CHƯƠNG 01: SPRING CORE & NỀN TẢNG SPRING BOOT

(Spring Core & Spring Boot Fundamentals)

---

1. INVERSION OF CONTROL (IoC) & DEPENDENCY INJECTION (DI)

1.1 Inversion of Control (IoC - Đảo ngược điều khiển)

- 🔹 **What is it? (Nó là gì?)**

- **Định nghĩa:** Inversion of Control (IoC) là một nguyên lý thiết kế phần mềm (Design Principle), trong đó quyền kiểm soát luồng thực thi và việc khởi tạo, quản lý vòng đời của các đối tượng (Objects/Dependencies) được chuyển giao từ lập trình viên (code của bạn) cho một bộ khung quản lý (Framework/Container) đảm nhiệm.
- **Tiếng Anh:** *Inversion of Control is a design principle in which the control of object creation, configuration, and lifecycle management is transferred from the application code to an IoC Container.*

- 🔹 **Why & When to use it? (Tại sao và Khi nào dùng?)**

- **Vấn đề khi KHÔNG có IoC (Truyền thống):**

  Khi một class `OrderService` cần gửi email thông báo, lập trình viên thường tự dùng toán tử `new`:

```
  public class OrderService {
private EmailService emailService;

public OrderService() {

// Tight coupling (Ràng buộc chặt chẽ)

this.emailService = new SmtpEmailService();

}

}
```

Hậu quả:

1. `OrderService` bị gắn chặt cứng ngắc với `SmtpEmailService`. Sau này muốn đổi sang `SendGridEmailService` hoặc `AwsSesEmailService` thì phải sửa đổi code của `OrderService` (vi phạm nguyên lý Open/Closed Principle trong SOLID).

2. Rất khó hoặc không thể viết Unit Test: Bạn không thể thay thế `emailService` bằng một bản Mock/Fake để kiểm thử độc lập mà không gửi email thật.

- **Lợi ích khi CÓ IoC:**

- **Loose Coupling (Giảm sự phụ thuộc):** Các class chỉ giao tiếp với nhau qua `Interface`, không quan tâm implementation cụ thể được tạo ra như thế nào.

- **Dễ dàng bảo trì và mở rộng:** Muốn đổi công nghệ chỉ cần cấu hình lại một bean duy nhất mà không cần sửa hàng chục Service khác.

- **Tối ưu hóa kiểm thử (Testability):** Dễ dàng inject Mock Object vào khi chạy Unit Test.

🔹 **How it works under the hood (Cơ chế hoạt động bên dưới)**

Trong Spring, **IoC Container** đóng vai trò là "nhà máy sản xuất và quản lý đối tượng".

Spring cung cấp 2 giao diện (Interface) chính đại diện cho IoC Container:

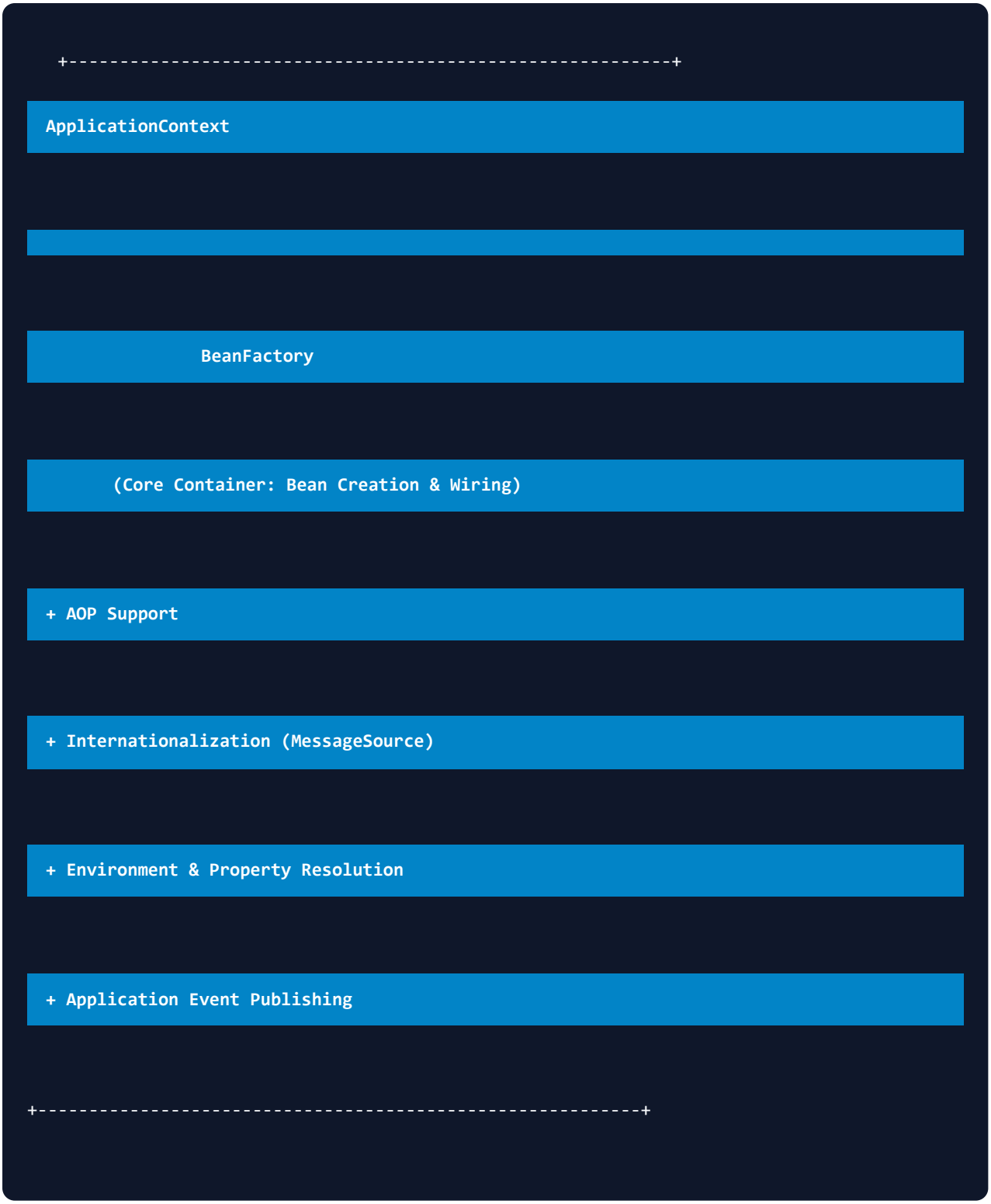
1. `BeanFactory`: Container cơ bản nhất, quản lý danh sách các Bean theo cơ chế *Lazy-loading* (chỉ khởi tạo bean khi có yêu cầu gọi `getBean()`). Rất nhẹ, thích hợp cho các thiết bị tài nguyên cực kỳ hạn chế.

2. `ApplicationContext`: Kế thừa từ `BeanFactory`, là container nâng cao được sử dụng trong 99.9% ứng dụng Spring thực tế. Hỗ trợ:

- *Eager-loading* (Khởi tạo sẵn tất cả các Singleton Beans ngay khi ứng dụng khởi động để phát hiện lỗi sớm).

- Tích hợp tính năng AOP (Aspect-Oriented Programming).

- Quản lý tài nguyên (Resource loading), đa ngôn ngữ (MessageSource / i18n), và xử lý sự kiện (Event publication).



1.2 Dependency Injection (DI - Tiêm phụ thuộc)

🔹 What is it? (Nó là gì?)

- **Định nghĩa:** Dependency Injection (DI) là một dạng hiện thực hóa cụ thể của nguyên lý IoC. Thay vì một đối tượng tự khởi tạo các đối tượng phụ thuộc của nó, Container sẽ "tiêm" (inject) trực tiếp các phụ thuộc đó vào đối tượng tại thời điểm khởi tạo (runtime).

🔹 3 Kiểu Dependency Injection trong Spring

Spring hỗ trợ 3 kiểu tiêm phụ thuộc. Bảng so sánh chi tiết:

Kiểu Injection	Cách viết	Đánh giá	Khuyên dùng?
1. Field Injection	Gắn `@Autowired` trực tiếp trên thuộc tính (field)	Vi phạm đóng gói, phụ thuộc Reflection, không thể tạo biến `final`, khó test độc lập không có Spring context.	❌ **Không khuyên dùng (Anti-pattern)**
2. Setter Injection	Gắn `@Autowired` trên hàm `set...()`	Phù hợp khi phụ thuộc là **tùy chọn (Optional)** hoặc có thể thay đổi lại trong runtime.	⚠️ **Chỉ dùng khi dependency là optional**
3. Constructor Injection	Tiêm qua Hàm khởi tạo (Constructor)	Đảm bảo tính bất biến (`final`), bắt buộc phải cung cấp đủ dependency mới khởi tạo được (Fail-fast), dễ viết Unit test độc lập (chỉ cần truyền mock vào constructor).	✅ **Chuẩn mực tốt nhất (Best Practice)**

🔹 Code minh họa & So sánh:

❌ Kiểu 1: Field Injection (Không nên dùng trong thực tế)

```
@Service

public class OrderService {


@Autowired // Spring dùng Reflection để gán giá trị vào private field


private PaymentGateway paymentGateway;


}
```

⚠️ Kiểu 2: Setter Injection

```
@Service

public class OrderService {


private NotificationService notificationService;


@Autowired


public void setNotificationService(NotificationService notificationService) {


this.notificationService = notificationService;


}


}
```

✅ **Kiểu 3: Constructor Injection (Chuẩn Enterprise)**

```
package com.example.service;

import org.springframework.stereotype.Service;


@Service


public class OrderService {


// 1. Dùng từ khóa 'final' để đảm bảo tính bất biến (Thread-safety & Immutability)


private final PaymentGateway paymentGateway;


private final EmailService emailService;


// 2. Từ Spring 4.3 trở đi, nếu class chỉ có 1 constructor DUY NHẤT,


//    bạn KHÔNG CẦN phải viết annotation @Autowired! Spring sẽ tự động hiểu và inject.


public OrderService(PaymentGateway paymentGateway, EmailService emailService) {


this.paymentGateway = paymentGateway;


this.emailService = emailService;


}


}
```

Mẹo nâng cao (Lombok Tip):

Để code ngắn gọn mà vẫn tuân thủ chuẩn Constructor Injection, các lập trình viên chuyên nghiệp luôn dùng `@RequiredArgsConstructor` của thư viện Lombok:

> [redacted]

> @Service

> @RequiredArgsConstructor // Lombok tự động sinh constructor cho tất cả các trường có từ khóa 'final'

> public class OrderService {

> private final PaymentGateway paymentGateway;

> private final EmailService emailService;

> }

>

---

2. SPRING BEANS & CÁC ANNOTATION NỀN TẢNG (STEREOTYPE ANNOTATIONS)

2.1 Spring Bean là gì?

- **Định nghĩa:** Một **Spring Bean** đơn giản là một Java Object được khởi tạo, lắp ráp (wiring), quản lý vòng đời và lưu trữ bên trong Spring IoC Container.
- Bất kỳ class nào được Spring quản lý thì instance của nó đều được gọi là một **Bean**.

2.2 Vòng đời của một Spring Bean (Bean Lifecycle)

Một Bean trong Spring trải qua các giai đoạn nghiêm ngặt từ khi sinh ra đến khi ứng dụng tắt:

```
graph TD

A["1. Nạp Bean Definition từ Source/Annotation"] --> B["2. Khởi tạo Instance (Instantiate - gọ


B --> C["3. Tiêm phụ thuộc (Populate Properties / Dependency Injection)"]


C --> D["4. Gọi BeanNameAware / ApplicationContextAware (nếu có)"]


D --> E["5. BeanPostProcessor: postProcessBeforeInitialization()"]


E --> F["6. Gọi hàm khởi tạo: @PostConstruct / InitializingBean"]


F --> G["7. BeanPostProcessor: postProcessAfterInitialization() (Tạo Proxy AOP nếu cần)"]


G --> H["8. BEAN SẴN SÀNG SỬ DỤNG (Ready for Use)"]


H --> I["9. Container Shutdown: Gọi hàm dọn dẹp @PreDestroy / DisposableBean"]
```

Code minh họa `@PostConstruct` và `@PreDestroy`:

```
package com.example.config;

import jakarta.annotation.PostConstruct;


import jakarta.annotation.PreDestroy;


import org.springframework.stereotype.Component;


@Component


public class CacheManager {


public CacheManager() {


System.out.println("Step 1: Constructor được gọi. Các dependency chưa được inject xong.");


}


@PostConstruct


public void init() {


// Chạy ngay sau khi Constructor và DI hoàn tất


System.out.println("Step 2: @PostConstruct được gọi. Khởi tạo kết nối Redis Cache hoặc nạp dữ


}


@PreDestroy


public void cleanup() {


// Chạy khi ứng dụng tắt (ApplicationContext.close())
```

```
System.out.println("Step 3: @PreDestroy được gọi. Giải phóng tài nguyên, đóng socket, đóng cac

}

}
```

2.3 Các Stereotype Annotations chính

Spring cung cấp các Annotation chuyên biệt để đánh dấu trách nhiệm (Layer) của từng class:

Annotation	Tầng áp dụng (Layer)	Ý nghĩa & Tính năng đặc biệt
`@Component`	Tầng chung (Generic)	Annotation gốc. Đánh dấu class là một Spring-managed component để cơ chế Component Scanning phát hiện.
`@Service`	Tầng nghiệp vụ (Service Layer)	Kế thừa từ `@Component`. Đánh dấu class chứa Business Logic của ứng dụng. Giúp mã nguồn rõ ràng theo kiến trúc phân tầng.
`@Repository`	Tầng truy xuất dữ liệu (DAO / Data Layer)	Kế thừa từ `@Component`. Đánh dấu class thao tác trực tiếp với Database. **Đặc biệt:** Tự động kích hoạt cơ chế dịch lỗi (Automatic Exception Translation), chuyển các lỗi SQL đặc thù của database thành nhánh ngoại lệ chung `DataAccessException` của Spring.
`@Controller`	Tầng hiển thị (MVC Controller)	Dành cho các ứng dụng Server-Side Rendering (trả về View HTML như Thymeleaf, JSP).
`@RestController`	Tầng Web API (RESTful API)	Kết hợp của `@Controller` + `@ResponseBody`. Mọi phương thức trong class này mặc định sẽ serialize kết quả trả về thành định dạng JSON/XML để gửi về cho client.
`@Configuration`	Tầng cấu hình (Configuration Layer)	Đánh dấu một class chứa các định nghĩa Bean tường minh (thông qua các method có `@Bean`). Spring sử dụng CGLIB Proxy để đảm bảo các method `@Bean` luôn trả về cùng một instance (Singleton pattern).

2.4 So sánh `@Component` vs `@Bean`

Đây là câu hỏi phỏng vấn kinh điển: *Khi nào dùng* `@Component`*, khi nào dùng* `@Bean` *?*

Tiêu chí	`@Component`	`@Bean`
Vị trí đặt	Đặt trên cấp độ **Class**	Đặt trên cấp độ **Method** (bên trong class `@Configuration`)
Quyền sở hữu Code	Chỉ dùng được khi **bạn sở hữu mã nguồn** của class đó (tự tay viết class).	Dùng khi muốn biến một class từ **thư viện bên thứ 3** (Third-party library không sửa code được) thành Spring Bean.
Khả năng tùy biến	Tự động quét và khởi tạo thông qua scan package.	Cho phép viết code tùy chỉnh phức tạp (set API key, timeout, credentials...) trước khi return object.

Code ví dụ thực tế sử dụng `@Configuration` và `@Bean`:

```
package com.example.config;

import com.fasterxml.jackson.databind.ObjectMapper;


import org.springframework.context.annotation.Bean;


import org.springframework.context.annotation.Configuration;


import org.springframework.web.client.RestClient;


@Configuration


public class AppConfig {


// 1. Đăng ký ObjectMapper của thư viện Jackson thành 1 Spring Bean


@Bean


public ObjectMapper objectMapper() {


ObjectMapper mapper = new ObjectMapper();


// Cấu hình các rule format ngày tháng, bỏ qua field null...


return mapper;


}


// 2. Đăng ký RestClient để gọi API bên thứ ba


@Bean


public RestClient paymentRestClient() {
```

```
return RestClient.builder()

.baseUrl("https://api.stripe.com/v1")

.defaultHeader("Authorization", "Bearer sk_test_12345")

.build();

}

}
```

---

3. SPECIAL BEANS & GIẢI QUYẾT XUNG ĐỘT PHỤ THUỘC (AMBIGUITY)

Khi bạn có 1 Interface nhưng lại có từ **2 Implementation** trở lên được đăng ký thành Bean, Spring sẽ không biết chọn Bean nào để inject và sẽ ném ra lỗi `NoUniqueBeanDefinitionException`.



Spring cung cấp 2 giải pháp chính: `@Primary` và `@Qualifier`.

3.1 `@Primary` (Ưu tiên mặc định)

- **Ý nghĩa:** Chỉ định một Bean là phương án ưu tiên hàng đầu nếu có nhiều Bean cùng loại mà không có chỉ dẫn cụ thể.

```
@Component
@Primary // Nếu không nói gì thêm, Spring sẽ tự động chọn Vnpay

public class VnpayPaymentService implements PaymentGateway {

@Override

public void processPayment(double amount) {

System.out.println("Processing via VNPay: " + amount);

}

}

@Component

public class MomoPaymentService implements PaymentGateway {

@Override

public void processPayment(double amount) {

System.out.println("Processing via Momo: " + amount);

}

}
```

3.2 `@Qualifier` (Chỉ định đích danh)

- **Ý nghĩa:** Chỉ định chính xác tên của Bean cần inject, ghi đè cả `@Primary`.

```
@Service
public class CheckoutService {

private final PaymentGateway paymentGateway;

// Sử dụng @Qualifier("momoPaymentService") để chỉ định đích danh dùng Momo

public CheckoutService(@Qualifier("momoPaymentService") PaymentGateway paymentGateway) {

this.paymentGateway = paymentGateway;

}

}
```

3.3 Bean Scopes (Phạm vi hoạt động của Bean)

Mặc định trong Spring, tất cả các Bean đều là **Singleton**. Bạn có thể thay đổi bằng annotation `@Scope`.

Tên Scope	Cơ chế hoạt động	Trường hợp sử dụng
`singleton` **(Mặc định)**	Chỉ có **duy nhất 1 instance** được tạo ra trong toàn bộ vòng đời của `ApplicationContext`. Mọi nơi inject đều nhận chung instance này.	Dùng cho 99% các Service, Repository, Controller (Yêu cầu: Class phải là **Stateless** - không lưu trạng thái riêng của từng request).
`prototype`	**Mỗi lần được inject hoặc gọi** `getBean()`, Spring sẽ tạo ra một instance hoàn toàn mới.	Dùng cho các đối tượng có lưu trữ trạng thái riêng biệt (Stateful objects).
`request`	Mỗi HTTP Request sẽ có một instance mới riêng biệt.	Chỉ dùng trong Web context (lưu thông tin audit, IP của từng request).
`session`	Mỗi phiên làm việc (HTTP Session) của 1 người dùng sẽ có 1 instance.	Lưu giỏ hàng (Shopping Cart) trong ứng dụng web truyền thống.

Code ví dụ:

```
@Component
@Scope("prototype") // hoặc @Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)

public class ReportGenerator {

private String reportId; // Stateful field

}
```

3.4 `@Lazy` (Khởi tạo trễ)

- **Mặc định:** Các Singleton Bean được Spring khởi tạo ngay lập tức khi ứng dụng vừa chạy (Eager initialization) để phát hiện lỗi sớm.
- **`@Lazy`:** Báo cho Spring biết: "*Chưa cần tạo instance của Bean này khi khởi động app. Hãy đợi đến khi có một request hoặc một service khác thực sự gọi đến nó thì mới tạo.*"
- **Khi nào dùng?** Khi Bean tiêu tốn rất nhiều tài nguyên khởi tạo (nạp AI model, đọc file cực lớn) nhưng lại hiếm khi được sử dụng.

```
@Component
@Lazy

public class HeavyReportService {

public HeavyReportService() {

System.out.println("HeavyReportService chỉ được khởi tạo khi có người dùng bấm xuất báo cáo!")

}

}
```

---

4. QUẢN LÝ CẤU HÌNH: PROPERTIES, YAML & SPRING PROFILES

4.1 Cấu hình với `application.yml`

Hiện nay, file định dạng YAML (`.yml`) được ưa chuộng hơn `.properties` truyền thống nhờ cấu trúc phân cấp cây rõ ràng, dễ đọc:

```
server:
port: 8080

servlet:

context-path: /api/v1

app:

jwt:

secret-key: 404E635266556A586E3272357538782F413F4428472B4B62506453675666B5970

expiration-ms: 86400000 # 1 ngày

mail:

support-email: support@mycompany.com
```

4.2 Nạp giá trị cấu hình vào Java Code

Cách 1: Dùng `@Value` (Thích hợp cho 1 vài giá trị đơn lẻ)

```
@Component

public class MailNotificationService {


@Value("${app.mail.support-email:default@example.com}")


private String supportEmail;


@Value("${server.port}")


private int serverPort;


}
```

Cách 2: Dùng `@ConfigurationProperties` (Chuẩn Type-Safe cho các cấu hình lớn)

```
package com.example.config;

import org.springframework.boot.context.properties.ConfigurationProperties;


import org.springframework.context.annotation.Configuration;


@Configuration


@ConfigurationProperties(prefix = "app.jwt")


public class JwtProperties {


private String secretKey;


private long expirationMs;


// Getters and Setters bắt buộc


public String getSecretKey() { return secretKey; }


public void setSecretKey(String secretKey) { this.secretKey = secretKey; }


public long getExpirationMs() { return expirationMs; }


public void setExpirationMs(long expirationMs) { this.expirationMs = expirationMs; }


}
```

4.3 Quản lý đa môi trường với Spring Profiles (`@Profile`)

Trong thực tế, bạn sẽ có các môi trường: `local` , `dev` , `staging` , `prod` . Mỗi môi trường sẽ kết nối một Database khác nhau, dùng cổng khác nhau.

Cấu trúc file cấu hình đa môi trường:

```
src/main/resources/
├── application.yml          # Cấu hình chung cho mọi môi trường

├── application-dev.yml      # Cấu hình riêng cho môi trường Development (dùng DB H2 hoặc loca

└── application-prod.yml     # Cấu hình riêng cho môi trường Production (dùng AWS RDS Postgres
```

`application.yml` **(File gốc):**

```
spring:
profiles:

active: dev # Mặc định kích hoạt profile 'dev' khi dev chạy local
```

`application-dev.yml` **:**

```
server:
port: 8080

spring:

datasource:

url: jdbc:postgresql://localhost:5432/my_dev_db

username: postgres

password: dev_password
```

`application-prod.yml`:

```
server:

port: 80


spring:


datasource:


url: jdbc:postgresql://${PROD_DB_HOST}:5432/my_prod_db


username: ${PROD_DB_USER}


password: ${PROD_DB_PASSWORD}
```

Cách kích hoạt profile khi triển khai (Production):

- Chạy bằng dòng lệnh Java:

```
  java -jar -Dspring.profiles.active=prod myapp.jar
```

- Hoặc dùng biến môi trường (Docker):

```
  SPRING_PROFILES_ACTIVE=prod
```

Sử dụng `@Profile` trên Java Class:

```
// Bean này CHỈ được khởi tạo khi đang ở môi trường DEV (ví dụ: tạo dữ liệu giả lập ban đầu)
@Component

@Profile("dev")

public class MockDataLoader implements CommandLineRunner {

@Override

public void run(String... args) {

System.out.println("Đang nạp 100 User mẫu vào cơ sở dữ liệu Dev...");

}

}
```

---

5. KIẾN TRÚC TỔNG THỂ 3 TẦNG (3-TIER LAYERED ARCHITECTURE)

Trong mọi ứng dụng Spring Boot chuẩn mực, luồng dữ liệu luôn đi theo một chiều nghiêm ngặt:


[HTTP Client / Postman / React / Mobile]
(JSON / HTTP Request)
Controller Layer
@RestController: Tiếp nhận request, Validate tham số đầu vào,
gọi Service và trả về ResponseEntity (HTTP Status + JSON DTO).
(DTO / Domain Call)
Service Layer
@Service: Trái tim của ứng dụng. Chứa toàn bộ Business Logic,
tính toán, xử lý @Transactional, gửi email, gọi API ngoài...
(Entities / Queries)
Repository Layer
@Repository / JpaRepository: Thao tác trực tiếp với Database.
Thực thi câu lệnh SQL / HQL / JPQL để CRUD dữ liệu.
(SQL Connection)



Quy tắc vàng:

- Controller **tuyệt đối không** được gọi trực tiếp Repository hay viết logic nghiệp vụ.

- Service **tuyệt đối không** nhận đối tượng `HttpServletRequest` hay xử lý mã HTTP Status (đó là việc của Controller).

- Mọi tầng phải độc lập và giao tiếp với nhau bằng DTO (Data Transfer Object).

CHƯƠNG 02: XÂY DỰNG RESTFUL WEB APIS CHUẨN QUỐC TẾ

(Building Production-Grade RESTful Web APIs, DTOs & Validation)

---

1. NỀN TẢNG KIẾN TRÚC REST & HTTP PROTOCOL

1.1 REST là gì? (Representational State Transfer)

- **Định nghĩa:** REST là một phong cách kiến trúc phần mềm định nghĩa các ràng buộc để xây dựng các dịch vụ web nhẹ, dễ mở rộng và độc lập nền tảng.
- **Tính chất quan trọng nhất: Statelessness (Phi trạng thái)** — Mỗi request từ client gửi lên server phải chứa đầy đủ toàn bộ thông tin cần thiết để xử lý request đó. Server không lưu trữ bất kỳ trạng thái phiên làm việc (session) nào của client trong bộ nhớ. Nhờ đó, hệ thống có thể dễ dàng scale ngang (Horizontal Scaling) bằng cách gắn thêm nhiều server đằng sau Load Balancer.

1.2 Bảng đối chiếu các HTTP Method chuẩn RESTful

Method	Mục đích	Tính An toàn (Safe)	Tính Bất biến (Idempotent)	Status Code trả về chuẩn
`GET`	Đọc / Truy vấn dữ liệu	✅ Có (Không sửa đổi dữ liệu)	✅ Có	`200 OK`
`POST`	Tạo mới tài nguyên (Resource)	❌ Không	❌ Không (gọi N lần tạo N bản ghi)	`201 Created` kèm Header `Location`
`PUT`	Cập nhật toàn bộ (Thay thế hoàn toàn tài nguyên)	❌ Không	✅ Có (gọi 1 hay 100 lần kết quả như nhau)	`200 OK` hoặc `204 No Content`
`PATCH`	Cập nhật một phần (Chỉ sửa 1-2 trường dữ liệu)	❌ Không	❌ Thường không (tùy logic)	`200 OK`
`DELETE`	Xóa tài nguyên	❌ Không	✅ Có (xóa 1 lần hay nhiều lần kết quả vẫn là tài nguyên đã mất)	`204 No Content` hoặc `200 OK`

> **Khái niệm Idempotent (Tính bất biến):**

> Một thao tác được gọi là Idempotent nếu việc thực thi nó **một lần** hay **nhiều lần liên tiếp** đều mang lại cùng một trạng thái kết quả trên hệ thống. Ví dụ: `DELETE /api/v1/users/10` chạy 1 lần hay 10 lần thì user 10 vẫn bị xóa khỏi database.

1.3 Các HTTP Status Code phổ biến phải ghi nhớ

```
1xx: Informational (Thông tin - hiếm khi dùng trực tiếp trong API)

2xx: Success (Thành công)


├── 200 OK: Thành công thông thường (GET, PUT, PATCH).


├── 201 Created: Tạo mới thành công (thường dùng cho POST).


└── 204 No Content: Thành công nhưng không có dữ liệu trả về trong body (thường dùng cho DELET


4xx: Client Error (Lỗi do phía người dùng / client gửi request sai)


├── 400 Bad Request: Dữ liệu gửi lên sai định dạng, thiếu trường bắt buộc.


├── 401 Unauthorized: Chưa xác thực (Chưa đăng nhập, thiếu JWT Token).


├── 403 Forbidden: Đã đăng nhập nhưng không có quyền truy cập (Ví dụ: User thường đòi vào tran


├── 404 Not Found: Không tìm thấy tài nguyên theo ID yêu cầu.


└── 409 Conflict: Xung đột dữ liệu (Ví dụ: Đăng ký tài khoản với email đã tồn tại).


5xx: Server Error (Lỗi nội bộ hệ thống / Bug code)


└── 500 Internal Server Error: Ngoại lệ không mong muốn chưa được xử lý (NullPointerException,
```

2. SPRING WEB ANNOTATIONS & ĐIỀU PHỐI REQUEST (ROUTING)

2.1 Tiếp nhận dữ liệu từ HTTP Request

Spring Web cung cấp các annotation để trích xuất dữ liệu từ các phần khác nhau của HTTP Request:

```
GET /api/v1/products/category/laptops?page=1&limit=10

Header: Authorization: Bearer eyJhbGci...


Body: { "name": "Dell XPS 15", "price": 1800.0 }
```

1. **`@PathVariable`** : Lấy giá trị nằm trực tiếp trên đường dẫn URL (URI Path). Thích hợp cho việc định danh duy nhất tài nguyên:

- URL: `/api/v1/users/{id}` ➔ `@PathVariable("id") Long userId`

2. **`@RequestParam`** : Lấy Query Parameters (nằm sau dấu `?` trên URL). Thích hợp cho bộ lọc (filter), tìm kiếm (search), phân trang (pagination):

- URL: `/api/v1/products?page=1&size=20` ➔ `@RequestParam(defaultValue = "0") int page`

3. **`@RequestBody`** : Đọc toàn bộ chuỗi JSON trong phần Body của HTTP Request và tự động giải mã (deserialize) thành đối tượng Java tương ứng thông qua thư viện Jackson.

4. **`@RequestHeader`** : Đọc thông tin từ Header của request (Token, User-Agent, API-Key...).

2.2 Mã nguồn hoàn chỉnh: Standard REST Controller

Dưới đây là một REST Controller hoàn chỉnh chuẩn công nghiệp quản lý `Product` :

```
package com.example.controller;

import com.example.dto.ProductCreateRequest;

import com.example.dto.ProductResponse;

import com.example.dto.ProductUpdateRequest;

import com.example.service.ProductService;

import jakarta.validation.Valid;

import lombok.RequiredArgsConstructor;

import org.springframework.data.domain.Page;

import org.springframework.data.domain.Pageable;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;

import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;

@RestController

@RequestMapping("/api/v1/products")

@RequiredArgsConstructor

public class ProductController {
```

```
private final ProductService productService;

// 1. GET: Lấy danh sách sản phẩm có phân trang & lọc

@GetMapping

public ResponseEntity<Page<ProductResponse>> getAllProducts(

@RequestParam(required = false) String keyword,

Pageable pageable

) {

Page<ProductResponse> products = productService.getProducts(keyword, pageable);

return ResponseEntity.ok(products); // Trả về HTTP 200 OK kèm payload

}

// 2. GET: Lấy chi tiết một sản phẩm theo ID

@GetMapping("/{id}")

public ResponseEntity<ProductResponse> getProductById(@PathVariable("id") Long id) {

ProductResponse product = productService.getProductById(id);

return ResponseEntity.ok(product);

}

// 3. POST: Tạo mới một sản phẩm
```

```
@PostMapping

public ResponseEntity<ProductResponse> createProduct(@Valid @RequestBody ProductCreateRequest

ProductResponse createdProduct = productService.createProduct(request);

// Chuẩn REST: Khi tạo mới (201 Created), luôn trả về Header 'Location' trỏ tới URL của tài ng

URI location = ServletUriComponentsBuilder

.fromCurrentRequest()

.path("/{id}")

.buildAndExpand(createdProduct.id())

.toUri();

return ResponseEntity.created(location).body(createdProduct);

}

// 4. PUT: Cập nhật thông tin sản phẩm

@PutMapping("/{id}")

public ResponseEntity<ProductResponse> updateProduct(

@PathVariable("id") Long id,

@Valid @RequestBody ProductUpdateRequest request

) {
```

```
ProductResponse updated = productService.updateProduct(id, request);

return ResponseEntity.ok(updated);

}

// 5. DELETE: Xóa sản phẩm

@DeleteMapping("/{id}")

public ResponseEntity<Void> deleteProduct(@PathVariable("id") Long id) {

productService.deleteProduct(id);

// Trả về HTTP 204 No Content vì không còn dữ liệu gì cần trả lại

return ResponseEntity.noContent().build();

}

}
```

---

3. DTO PATTERN (DATA TRANSFER OBJECT) & ÁNH XẠ MAPSTRUCT

3.1 Tại sao TUYỆT ĐỐI KHÔNG ĐƯỢC trả trực tiếp Entity ra ngoài API?

Trong rất nhiều video tutorial sơ sài, người dạy thường viết:

```
// ❌ CỰC KỲ NGUY HIỂM TRONG THỰC TẾ:
@GetMapping("/{id}")

public User getUser(@PathVariable Long id) {

return userRepository.findById(id).get(); // Trả thẳng Database Entity User ra JSON!

}
```

🚨 **3 Hậu quả nghiêm trọng nếu trả thẳng Entity ra ngoài:**

1. **Lộ thông tin nhạy cảm (Security Breach):** Entity `User` chứa các trường như `passwordHash`, `resetPasswordToken`, `socialSecurityNumber`. Khi Jackson serialize, toàn bộ thông tin này sẽ lộ cho người dùng xem trên trình duyệt!

2. **Lỗi vòng lặp vô tận (Infinite Recursion / StackOverflowError):** Trong quan hệ 2 chiều giữa các Entity (ví dụ: `Order` có danh sách `OrderItem`, mỗi `OrderItem` lại trỏ ngược về `Order`), Jackson sẽ serialize lặp đi lặp lại giữa 2 đối tượng cho đến khi tràn bộ nhớ stack.

3. **Phá vỡ tính đóng gói của Database:** Bất kỳ thay đổi cấu trúc bảng nào trong DB cũng sẽ ngay lập tức làm vỡ hợp đồng API với Frontend/Mobile app.

4. **Vấn đề Over-fetching:** Bạn kéo toàn bộ 30 cột trong bảng DB lên chỉ để hiển thị 2 trường `id` và `name` trên màn hình danh sách.

👉 **Giải pháp:** Luôn sử dụng **DTO (Data Transfer Object)** để giao tiếp với Client.

3.2 Hiện thực DTO hiện đại với Java 17+ `record`

Từ Java 16/17+, `record` là lựa chọn số 1 để làm DTO:

- Cú pháp siêu ngắn gọn (không cần viết getter, setter, `equals()`, `hashCode()`, `toString()`).
- Tính chất **Immutable (Bất biến)**: Các trường mặc định là `private final`, đảm bảo an toàn tuyệt đối khi truyền tải dữ liệu giữa các luồng.

```
package com.example.dto;

import jakarta.validation.constraints.NotBlank;

import jakarta.validation.constraints.NotNull;

import jakarta.validation.constraints.Positive;

import java.math.BigDecimal;

// DTO dùng cho request tạo mới sản phẩm

public record ProductCreateRequest(

@NotBlank(message = "Tên sản phẩm không được để trống")

String name,

String description,

@NotNull(message = "Giá không được để trống")

@Positive(message = "Giá sản phẩm phải lớn hơn 0")

BigDecimal price,

@NotNull(message = "Danh mục không được để trống")

Long categoryId

) {}
```

```
package com.example.dto;

import java.math.BigDecimal;


import java.time.Instant;


// DTO dùng cho response trả về cho client


public record ProductResponse(


Long id,


String name,


String description,


BigDecimal price,


String categoryName,


Instant createdAt


) {}
```

3.3 Ánh xạ tự động hiệu năng cao với MapStruct

Thay vì phải tự tay gõ `productResponse.setName(entity.getName())` lặp đi lặp lại hàng trăm lần, ta sử dụng thư viện **MapStruct**:

- **Cơ chế hoạt động:** MapStruct sinh ra code Java thuần túy tại thời điểm biên dịch (Compile-time code generation). Nó **không dùng Reflection** trong runtime nên tốc độ nhanh ngang ngửa code tự viết tay!

Cấu hình Maven trong `pom.xml`:

```
<dependencies>

<dependency>

<groupId>org.mapstruct</groupId>

<artifactId>mapstruct</artifactId>

<version>1.5.5.Final</version>

</dependency>

</dependencies>

<build>

<plugins>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.11.0</version>

<configuration>

<annotationProcessorPaths>

<!-- Lưu ý: Nếu có Lombok, phải đặt Lombok trước MapStruct Binding -->

<path>
```

```
<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

<version>1.18.30</version>

</path>

<path>

<groupId>org.mapstruct</groupId>

<artifactId>mapstruct-processor</artifactId>

<version>1.5.5.Final</version>

</path>

</annotationProcessorPaths>

</configuration>

</plugin>

</plugins>

</build>
```

Định nghĩa Interface Mapper:

```
package com.example.mapper;

import com.example.dto.ProductCreateRequest;


import com.example.dto.ProductResponse;


import com.example.entity.Product;


import org.mapstruct.Mapper;


import org.mapstruct.Mapping;


@Mapper(componentModel = "spring") // Đăng ký mapper này thành 1 Spring Bean


public interface ProductMapper {


// Ánh xạ từ Entity sang Response DTO


@Mapping(source = "category.name", target = "categoryName") // Lấy tên từ quan hệ Category gán


ProductResponse toResponse(Product product);


// Ánh xạ từ Request DTO sang Entity để lưu Database


@Mapping(target = "id", ignore = true) // ID do Database tự sinh nên bỏ qua


@Mapping(target = "category", ignore = true) // Category sẽ được query và gán riêng trong Serv


@Mapping(target = "createdAt", ignore = true)


Product toEntity(ProductCreateRequest request);
```

```
}
```

---

4. XÁC THỰC DỮ LIỆU ĐẦU VÀO (DATA VALIDATION)

Để đảm bảo an toàn hệ thống, dữ liệu từ Client gửi lên phải được kiểm tra tính hợp lệ ngay tại tầng Controller trước khi đưa vào Service.

4.1 Thư viện `spring-boot-starter-validation`

Khai báo trong `pom.xml`:

```
<dependency>
<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-validation</artifactId>

</dependency>
```

4.2 Bảng các Annotation Validation thông dụng nhất

Annotation	Mục đích kiểm tra	Ví dụ áp dụng
`@NotNull`	Giá trị không được là `null` (nhưng có thể là chuỗi rỗng `""`).	`Long id`, `BigDecimal price`
`@NotEmpty`	Không `null` và kích thước/độ dài phải > 0.	`List tags`, `String code`
`@NotBlank`	Không `null`, không rỗng và **không chứa toàn ký tự khoảng trắng**.	`String username`, `String title`
`@Size(min=x, max=y)`	Giới hạn độ dài chuỗi hoặc số lượng phần tử mảng.	`@Size(min = 6, max = 32)`
`@Min(value)` / `@Max(value)`	Giới hạn giá trị số nguyên tối thiểu / tối đa.	`@Min(18)` cho tuổi

`@Positive` / `@PositiveOrZero`	Giá trị số phải lớn hơn 0 hoặc >= 0.	Số lượng tồn kho, giá tiền
`@Email`	Phải đúng định dạng địa chỉ email hợp lệ.	`user@company.com`
`@Pattern(regexp = "...")`	Kiểm tra chuỗi khớp với biểu thức chính quy (Regex).	Số điện thoại, mật khẩu phức tạp
`@Past` / `@Future`	Kiểm tra ngày tháng phải ở quá khứ hoặc tương lai.	Ngày sinh nhật (`@Past`), Hạn visa (`@Future`)

4.3 Kích hoạt Validation với `@Valid`

- Bắt buộc phải đặt `@Valid` trước `@RequestBody` trong Controller method. Nếu thiếu `@Valid`, Spring sẽ bỏ qua toàn bộ các annotation kiểm tra bên trong DTO!

```
@PostMapping
public ResponseEntity<Void> registerUser(@Valid @RequestBody UserRegisterRequest request) {

userService.register(request);

return ResponseEntity.status(HttpStatus.CREATED).build();

}
```

---

5. XỬ LÝ NGOẠI LỆ TẬP TRUNG (GLOBAL EXCEPTION HANDLING)

5.1 Vấn đề với cách try-catch thông thường

Nếu trong mỗi Controller method bạn đều viết:

```
try {
userService.updateUser(id, request);

} catch (UserNotFoundException e) {

return ResponseEntity.status(404).body(...);

} catch (Exception e) {

return ResponseEntity.status(500).body(...);

}
```

Mã nguồn của bạn sẽ bị ô nhiễm bởi hàng trăm khối `try-catch` trùng lặp, vi phạm nguyên lý DRY (Don't Repeat Yourself).

5.2 Giải pháp chuẩn: `@RestControllerAdvice` & Chuẩn RFC 7807

- `@RestControllerAdvice` : Là một Aspect (AOP) đánh chặn (intercept) tất cả các Exception được ném ra từ **bất kỳ Controller nào** trong toàn bộ hệ thống.
- Từ Spring Boot 3, Spring chính thức hỗ trợ lớp `ProblemDetail` tuân thủ tiêu chuẩn quốc tế **RFC 7807 (Problem Details for HTTP APIs)**.

1. Định nghĩa Ngoại lệ Nghiệp vụ riêng (Custom Business Exceptions):

```
package com.example.exception;

// Ngoại lệ khi không tìm thấy tài nguyên (Resource Not Found)

public class ResourceNotFoundException extends RuntimeException {

public ResourceNotFoundException(String message) {

super(message);

}

}
```

```
package com.example.exception;

// Ngoại lệ khi tài nguyên đã tồn tại hoặc xung đột dữ liệu (Conflict)

public class DuplicateResourceException extends RuntimeException {

public DuplicateResourceException(String message) {

super(message);

}

}
```

2. Viết Global Exception Handler hoàn chỉnh:

```
package com.example.exception;

import org.springframework.http.HttpStatus;

import org.springframework.http.ProblemDetail;

import org.springframework.http.ResponseEntity;

import org.springframework.validation.FieldError;

import org.springframework.web.bind.MethodArgumentNotValidException;

import org.springframework.web.bind.annotation.ExceptionHandler;

import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.net.URI;

import java.time.Instant;

import java.util.HashMap;

import java.util.Map;

@RestControllerAdvice

public class GlobalExceptionHandler {

// 1. Xử lý lỗi 404: ResourceNotFoundException

@ExceptionHandler(ResourceNotFoundException.class)

public ResponseEntity<ProblemDetail> handleResourceNotFound(ResourceNotFoundException ex) {
```

```
ProblemDetail problemDetail = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMes

problemDetail.setTitle("Tài nguyên không tồn tại");

problemDetail.setType(URI.create("https://api.mycompany.com/errors/not-found"));

problemDetail.setProperty("timestamp", Instant.now());

return ResponseEntity.status(HttpStatus.NOT_FOUND).body(problemDetail);

}

// 2. Xử lý lỗi 409: DuplicateResourceException (Trùng lặp email, username...)

@ExceptionHandler(DuplicateResourceException.class)

public ResponseEntity<ProblemDetail> handleDuplicateResource(DuplicateResourceException ex) {

ProblemDetail problemDetail = ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, ex.getMess

problemDetail.setTitle("Dữ liệu bị xung đột");

problemDetail.setProperty("timestamp", Instant.now());

return ResponseEntity.status(HttpStatus.CONFLICT).body(problemDetail);

}

// 3. Xử lý lỗi 400: Dữ liệu Validation thất bại (@Valid bắn ra MethodArgumentNotValidExceptio

@ExceptionHandler(MethodArgumentNotValidException.class)

public ResponseEntity<ProblemDetail> handleValidationErrors(MethodArgumentNotValidException ex
```

```
ProblemDetail problemDetail = ProblemDetail.forStatusAndDetail(

HttpStatus.BAD_REQUEST,

"Một hoặc nhiều trường dữ liệu không hợp lệ."

);

problemDetail.setTitle("Lỗi xác thực dữ liệu đầu vào");

problemDetail.setProperty("timestamp", Instant.now());

// Gom tất cả các lỗi của từng trường vào 1 Map: { "email": "Email không đúng định dạng", "pri

Map<String, String> fieldErrors = new HashMap<>();

for (FieldError error : ex.getBindingResult().getFieldErrors()) {

fieldErrors.put(error.getField(), error.getDefaultMessage());

}

problemDetail.setProperty("invalidFields", fieldErrors);

return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(problemDetail);

}

// 4. Bắt tất cả các lỗi không mong muốn còn lại (500 Internal Server Error)

@ExceptionHandler(Exception.class)

public ResponseEntity<ProblemDetail> handleGlobalInternalError(Exception ex) {
```

```
// Ghi log lỗi để dev điều tra

ex.printStackTrace();

ProblemDetail problemDetail = ProblemDetail.forStatusAndDetail(

HttpStatus.INTERNAL_SERVER_ERROR,

"Đã có lỗi xảy ra phía hệ thống. Vui lòng liên hệ quản trị viên."

);

problemDetail.setTitle("Lỗi máy chủ nội bộ");

problemDetail.setProperty("timestamp", Instant.now());

return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(problemDetail);

}

}
```

3. Kết quả JSON trả về cho Client khi bị lỗi Validation (Đẹp, rõ ràng và chuẩn RFC):

```
{
"type": "about:blank",

"title": "Lỗi xác thực dữ liệu đầu vào",

"status": 400,

"detail": "Một hoặc nhiều trường dữ liệu không hợp lệ.",

"instance": "/api/v1/products",

"timestamp": "2026-09-05T13:20:00.000Z",

"invalidFields": {

"name": "Tên sản phẩm không được để trống",

"price": "Giá sản phẩm phải lớn hơn 0"

}

}
```

Client Frontend (React/Flutter) có thể đọc ngay `invalidFields` để highlight viền đỏ trên ô nhập liệu của người dùng một cách chính xác tuyệt đối.

CHƯƠNG 03: LƯU TRỮ DỮ LIỆU VỚI SPRING DATA JPA & DATABASES

(Persistence Layer: JPA, Hibernate, Relationships, Querying & NoSQL)

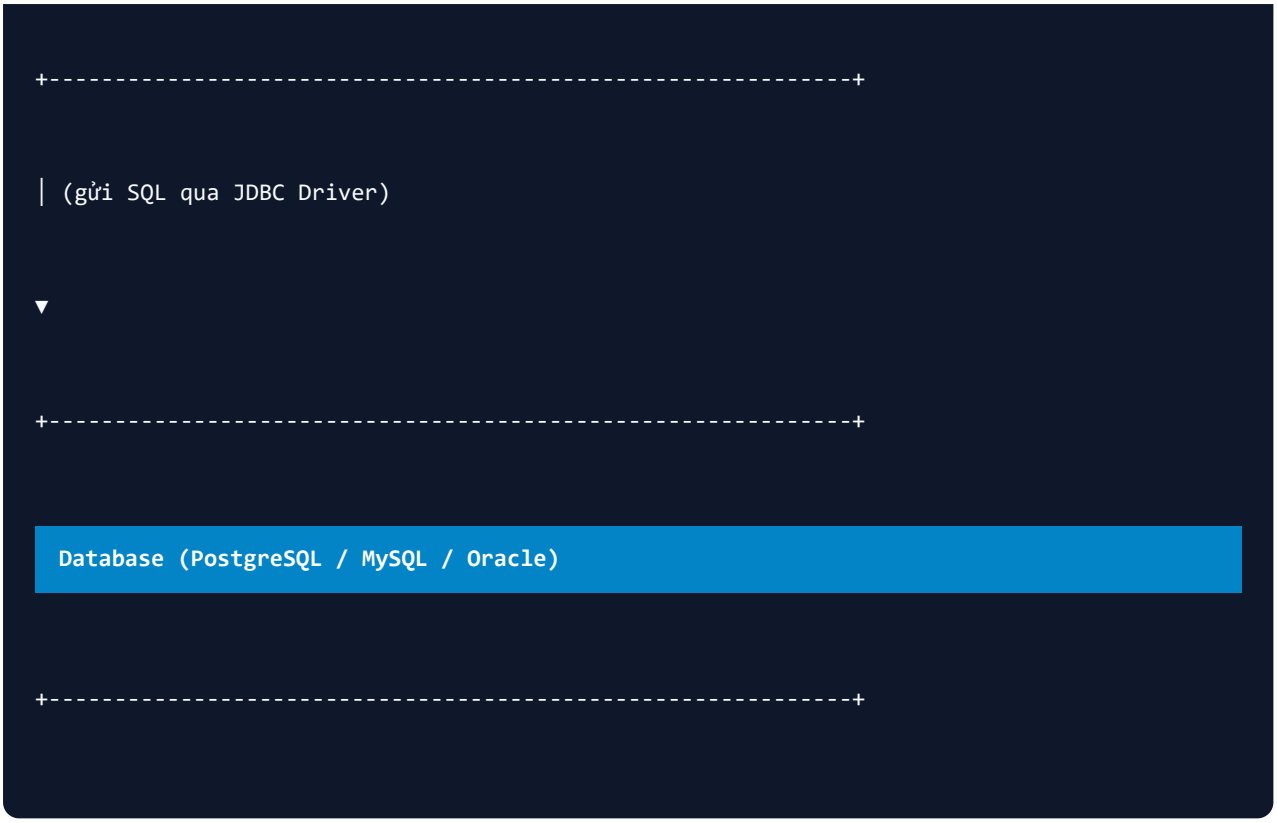
---

1. BẢN CHẤT CỐT LÕI CỦA ORM, JPA VÀ HIBERNATE

1.1 Khái niệm cơ bản (What is it?)

- **ORM (Object-Relational Mapping):** Là kỹ thuật lập trình ánh xạ giữa thế giới hướng đối tượng trong mã nguồn (Java Classes, Objects) với thế giới dữ liệu quan hệ trong cơ sở dữ liệu (Tables, Columns, Foreign Keys).
- **JPA (Jakarta Persistence API):** Là một **Bộ đặc tả tiêu chuẩn (Specification)** của Java. Nó chỉ chứa các `interface`, định nghĩa các `annotation` (`@Entity`, `@Table`, `@Id` ...) mà **không chứa code thực thi bên dưới**.
- **Hibernate:** Là một thư viện **Hiện thực hóa (Implementation / Provider)** tiêu chuẩn JPA. Hibernate chính là công cụ thực sự sinh ra các câu lệnh SQL (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) để gửi xuống database.
- **Spring Data JPA:** Là một tầng trừu tượng (Abstraction Layer) cao hơn do Spring phát triển, bọc lấy JPA/Hibernate, giúp lập trình viên chỉ cần khai báo một `interface` kế thừa `JpaRepository` là đã có sẵn hàng chục hàm CRUD mà không cần viết dù chỉ một dòng code SQL hay code cấu hình EntityManager!


Spring Data JPA
(Cung cấp JpaRepository, Derived Queries, Pagination...)
(sử dụng)
Jakarta Persistence (JPA)
(Tập hợp các Interface & Annotation chuẩn: @Entity, @Id...)
(hiện thực hóa)
Hibernate
(Thư viện ORM Engine: Sinh câu lệnh SQL, Dirty Checking)



1.2 Persistence Context & Cơ chế Dirty Checking kỳ diệu

Persistence Context là gì?

- **Persistence Context** là một vùng nhớ tạm (Cache cấp 1 - First-level Cache) do Hibernate quản lý trong suốt vòng đời của một Transaction.
- Mọi Entity được đọc từ Database lên đều được đưa vào trạng thái **Managed (Được quản lý)** trong Persistence Context.

Cơ chế Dirty Checking (Tự động phát hiện thay đổi)

- **Câu hỏi phỏng vấn kinh điển:** *"Tại sao trong phương thức có `@Transactional`, tôi chỉ gọi `user.setEmail("new@gmail.com")` mà không hề gọi `userRepository.save(user)`, dữ liệu trong Database vẫn tự động được cập nhật?"*
- **Cơ chế hoạt động:**

  1. Khi Hibernate đọc bản ghi từ DB lên, nó lưu một bản sao chụp nhanh (Snapshot) của Entity đó vào Persistence Context.

  2. Khi bạn thay đổi thuộc tính qua hàm setter, đối tượng trong bộ nhớ bị thay đổi.

  3. Tại thời điểm kết thúc Transaction (commit), Hibernate tự động so sánh trạng thái hiện tại của Entity với bản Snapshot ban đầu.

  4. Nếu phát hiện có sự sai khác (Dirty), Hibernate sẽ **tự động phát sinh câu lệnh SQL `UPDATE`** và gửi xuống Database trước khi commit!

```
graph TD

A["Bắt đầu @Transactional"] --> B["userRepository.findById(1L)"]


B --> C["Hibernate truy vấn SQL SELECT và đưa User vào Persistence Context (Lưu 1 bản Snapshot

C --> D["user.setStatus(ACTIVE) -> Dữ liệu đối tượng thay đổi"]


D --> E["Kết thúc hàm @Transactional"]


E --> F["Hibernate so sánh User hiện tại với Snapshot (Dirty Checking) -> Phát hiện khác biệt!

F --> G["Tự động sinh SQL: UPDATE users SET status = 'ACTIVE' WHERE id = 1"]


G --> H["Commit Transaction"]
```

---

2. ÁNH XẠ THỰC THỂ (ENTITY MAPPING) CHUẨN ENTERPRISE

2.1 Bảng các Annotation Ánh xạ thuộc tính

Annotation	Ý nghĩa & Cấu hình quan trọng	Lưu ý thực tế
`@Entity`	Đánh dấu class là một thực thể JPA được ánh xạ với một bảng DB.	Bắt buộc phải có Constructor không tham số (`no-args constructor`).
`@Table(name = "...")`	Tùy chỉnh tên bảng trong DB.	Nên đặt tên bảng theo số nhiều chữ thường (ví dụ: `tbl_users`, `products`).
`@Id`	Đánh dấu trường làm Khóa chính (Primary Key).	Bắt buộc mọi Entity phải có khóa chính.

`@GeneratedValue`	Chiến lược sinh khóa chính tự động.	`GenerationType.IDENTITY`: Tận dụng `AUTO_INCREMENT` của MySQL / PostgreSQL. `GenerationType.SEQUENCE`: Tận dụng Sequence của Postgres/Oracle. `GenerationType.UUID`: Sinh chuỗi UUID 128-bit duy nhất toàn cầu.
`@Column`	Tùy chỉnh cột (`name`, `nullable`, `unique`, `length`, `updatable`).	Với các trường không bao giờ được sửa (như `created_at`), đặt `updatable = false`.
`@Enumerated`	Ánh xạ kiểu Java `enum`.	**LUÔN DÙNG:** `@Enumerated(EnumType.STRING)` để lưu tên chữ (VD: `'ACTIVE'`). ❌ **TUYỆT ĐỐI KHÔNG DÙNG:** `EnumType.ORDINAL` vì nếu thêm giá trị enum mới vào giữa, toàn bộ ID trong DB sẽ bị sai lệch hoàn toàn!

2.2 Tự động ghi nhận thời gian với Spring Data JPA Auditing

Trong mọi dự án thực tế, mọi bảng đều cần 2 trường: `created_at` (ngày tạo) và `updated_at` (ngày sửa đổi cuối cùng). Ta có thể kích hoạt cơ chế tự động điền mà không cần code tay:

1. Kích hoạt Auditing trong `@Configuration`:

```
@Configuration
@EnableJpaAuditing // Kích hoạt tính năng tự động ghi nhận thời gian

public class JpaConfig {}
```

2. Viết Entity hoàn chỉnh:

```
package com.example.entity;

import jakarta.persistence.*;


import lombok.*;


import org.springframework.data.annotation.CreatedDate;


import org.springframework.data.annotation.LastModifiedDate;


import org.springframework.data.jpa.domain.support.AuditingEntityListener;


import java.math.BigDecimal;


import java.time.Instant;


@Entity


@Table(name = "products", uniqueConstraints = {


@UniqueConstraint(name = "uk_product_sku", columnNames = "sku")


})


@EntityListeners(AuditingEntityListener.class) // Lắng nghe sự kiện để tự điền ngày tháng


@Getter


@Setter


@NoArgsConstructor


@AllArgsConstructor
```

```
@Builder

public class Product {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private Long id;

@Column(name = "sku", nullable = false, length = 50, updatable = false)

private String sku; // Mã quản lý hàng hóa duy nhất

@Column(name = "name", nullable = false, length = 200)

private String name;

@Column(name = "price", nullable = false, precision = 12, scale = 2)

private BigDecimal price;

@Enumerated(EnumType.STRING)

@Column(name = "status", nullable = false, length = 20)

private ProductStatus status;

@CreatedDate

@Column(name = "created_at", nullable = false, updatable = false)

private Instant createdAt;
```

```
@LastModifiedDate

@Column(name = "updated_at", nullable = false)

private Instant updatedAt;

}
```

---

3. CÁC MỐI QUAN HỆ TRONG DATABASE (ENTITY RELATIONSHIPS)

Trong JPA, có 4 mối quan hệ cốt lõi:

1. `@ManyToOne` (Nhiều - Một): Mối quan hệ phổ biến nhất (90% các bài toán).
2. `@OneToMany` (Một - Nhiều): Phía ngược lại của `@ManyToOne`.
3. `@OneToOne` (Một - Một): Ví dụ: User với UserProfile.
4. `@ManyToMany` (Nhiều - Nhiều): Ví dụ: Student với Course.

3.1 Mối quan hệ `@ManyToOne` và `@OneToMany` (Chuẩn mực 2 chiều)

Hãy xem mối quan hệ giữa **Category (Danh mục)** và **Product (Sản phẩm)**:

- 1 Danh mục có **Nhiều** Sản phẩm (`@OneToMany`).
- 1 Sản phẩm thuộc về **Một** Danh mục (`@ManyToOne`).

Quy tắc Owning Side (Bên sở hữu khóa ngoại):

- Trong cơ sở dữ liệu quan hệ, bảng nào chứa cột **Khóa ngoại (Foreign Key)** thì Entity tương ứng bên Java sẽ là **Owning Side**.
- Ở đây, bảng `products` chứa cột `category_id`, do đó class `Product` là **Owning Side**. Phía `Category` là **Inverse Side** (bên được tham chiếu) và bắt buộc phải khai báo `mappedBy`.

Code mẫu thực chiến:

Phía Owning Side (`Product.java`):

```
@Entity

@Table(name = "products")


@Getter


@Setter


@NoArgsConstructor


public class Product {


@Id


@GeneratedValue(strategy = GenerationType.IDENTITY)


private Long id;


private String name;


// QUAN TRỌNG: Luôn đặt FetchType.LAZY để tránh kéo theo toàn bộ Category khi chỉ cần đọc Prod


@ManyToOne(fetch = FetchType.LAZY, optional = false)


@JoinColumn(name = "category_id", nullable = false, foreignKey = @ForeignKey(name = "fk_produc


private Category category;


}
```

Phía Inverse Side (`Category.java`):

```
@Entity
@Table(name = "categories")

@Getter

@Setter

@NoArgsConstructor

public class Category {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private Long id;

private String name;

// mappedBy trỏ tới tên biến 'category' bên trong class Product

// cascade = CascadeType.ALL: Khi lưu/xóa Category thì tự động lưu/xóa luôn các Product con

// orphanRemoval = true: Nếu xóa 1 Product khỏi danh sách products, nó sẽ bị xóa khỏi Database

@OneToMany(mappedBy = "category", cascade = CascadeType.ALL, orphanRemoval = true, fetch = Fet

private List<Product> products = new ArrayList<>();

// Helper Methods: Đồng bộ cả 2 đầu của mối quan hệ trong bộ nhớ RAM

public void addProduct(Product product) {

products.add(product);
```

```
product.setCategory(this);

}

public void removeProduct(Product product) {

products.remove(product);

product.setCategory(null);

}

}
```

3.2 FetchType: LAZY vs EAGER

Loại Fetch	Cơ chế	Đánh giá
`FetchType.EAGER`	Tải dữ liệu **ngay lập tức**. Khi bạn query Product, Hibernate sẽ tự động `LEFT OUTER JOIN` để tải luôn cả Category và các bảng liên quan.	❌ **Cực kỳ nguy hiểm!** Làm chậm hệ thống, tiêu tốn RAM và là nguyên nhân chính gây ra lỗi tràn bộ nhớ khi dữ liệu lớn. Mặc định `@ManyToOne` và `@OneToOne` trong JPA lại là EAGER, vì vậy bạn **phải luôn luôn ghi đè thành LAZY**.
`FetchType.LAZY`	Tải dữ liệu **theo yêu cầu (chỉ khi nào gọi mới tải)**. Khi bạn query Product, Hibernate chỉ gán một đối tượng Proxy rỗng vào trường Category. Chỉ khi nào code của bạn gọi `product.getCategory().getName()`, Hibernate mới thực sự bắn thêm câu lệnh SQL xuống DB để lấy dữ liệu.	✅ **Best Practice:** 100% các quan hệ đều phải đặt là `FetchType.LAZY`.

3.3 HIỂM HỌA LỖI N+1 QUERIES & CÁCH HÓA GIẢI TRIỆT ĐỂ

Lỗi N+1 Queries phát sinh như thế nào?

Giả sử trong database có **100 sản phẩm** thuộc các danh mục khác nhau. Bạn viết code:

```
List<Product> products = productRepository.findAll(); // Câu truy vấn số 1: SELECT * FROM prod

for (Product p : products) {


System.out.println(p.getCategory().getName()); // Phát sinh thêm N = 100 câu lệnh SELECT categ


}
```

👉 **Tổng cộng: 1 + 100 = 101 câu truy vấn SQL gửi xuống Database chỉ để in 100 dòng text!**

Nếu có 10.000 sản phẩm, ứng dụng sẽ gửi 10.001 query làm tê liệt hoàn toàn database server.

✅ **Cách giải quyết 1: Sử dụng `JOIN FETCH` trong JPQL**

Sử dụng từ khóa `FETCH` để bảo Hibernate kết hợp bảng (INNER/LEFT JOIN) và nạp toàn bộ dữ liệu quan hệ trong **1 câu truy vấn duy nhất**:

```
@Repository

public interface ProductRepository extends JpaRepository<Product, Long> {


@Query("SELECT p FROM Product p JOIN FETCH p.category WHERE p.status = :status")


List<Product> findAllWithCategoryByStatus(@Param("status") ProductStatus status);


}
```

✅ **Cách giải quyết 2: Sử dụng `@EntityGraph` (Phong cách hiện đại của Spring Data JPA)**

Không cần tự gõ JPQL nối chuỗi, chỉ cần đặt `@EntityGraph` trên hàm repository:

```
@Repository

public interface ProductRepository extends JpaRepository<Product, Long> {


// Spring Data JPA sẽ tự động thêm LEFT JOIN FETCH để nạp trường category


@EntityGraph(attributePaths = {"category"})

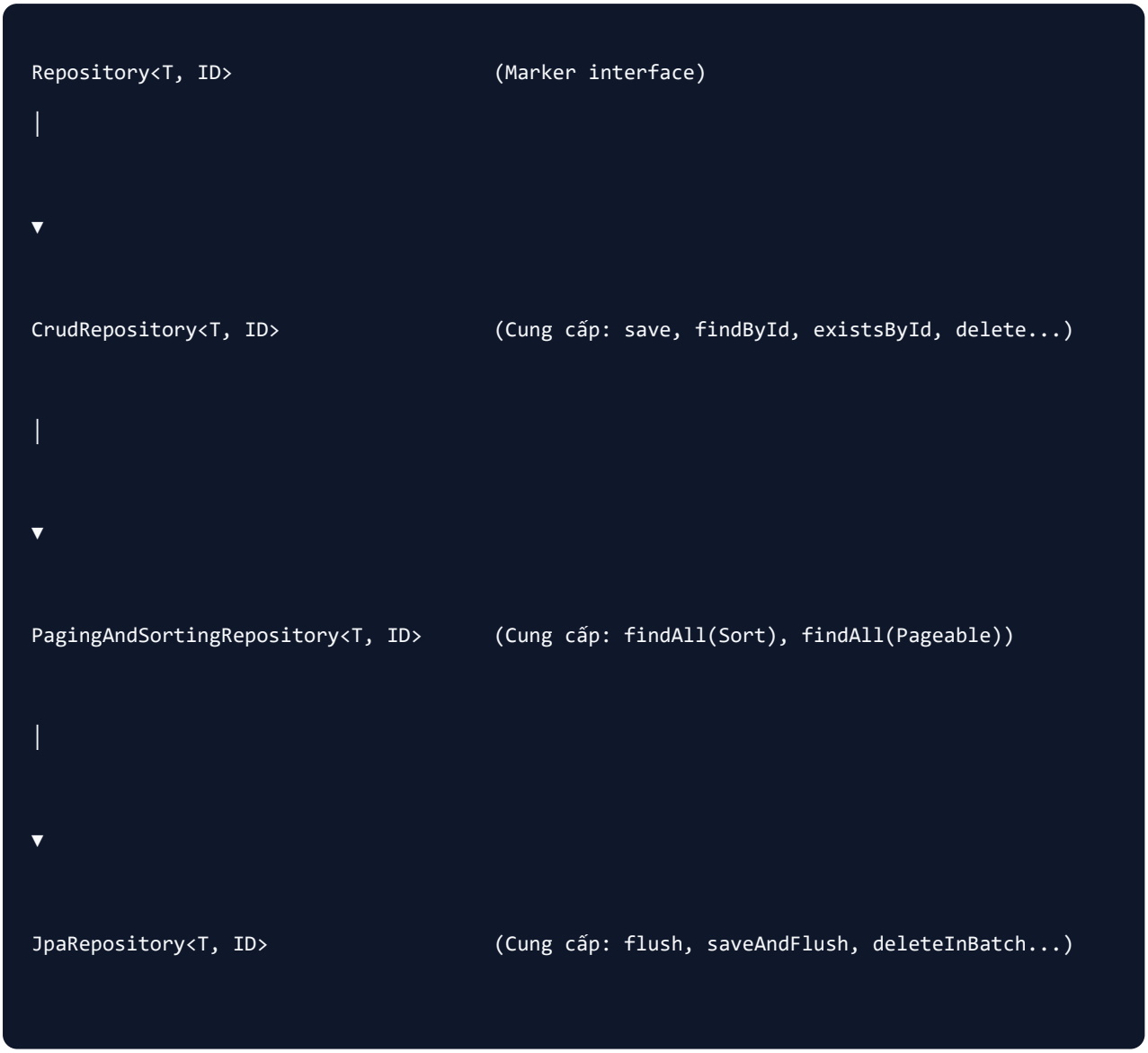

List<Product> findByNameContaining(String name);


}
```

---

4. SPRING DATA JPA REPOSITORIES & TRUY VẤN NÂNG CAO

4.1 Cây phả hệ Repository



👉 Trong mọi dự án, ta luôn cho interface của mình kế thừa `JpaRepository`.

4.2 Derived Query Methods (Tự động sinh câu lệnh từ tên hàm)

Spring Data JPA có khả năng phân tích cú pháp tên hàm và tự động chuyển thành câu lệnh SQL tương ứng:

```
public interface UserRepository extends JpaRepository<User, Long> {
// SELECT * FROM users WHERE email = ?

Optional<User> findByEmail(String email);

// SELECT CASE WHEN COUNT(*) > 0 THEN TRUE ELSE FALSE END FROM users WHERE email = ?

boolean existsByEmail(String email);

// SELECT * FROM users WHERE LOWER(full_name) LIKE LOWER(CONCAT('%', ?, '%'))

List<User> findByFullNameContainingIgnoreCase(String keyword);

// SELECT * FROM users WHERE age >= ? AND status = ?

List<User> findByAgeGreaterThanEqualAndStatus(int age, UserStatus status);

// SELECT * FROM users WHERE created_at BETWEEN ? AND ? ORDER BY created_at DESC

List<User> findByCreatedAtBetweenOrderByCreatedAtDesc(Instant start, Instant end);

}
```

4.3 JPQL vs Native SQL Query

Tiêu chí	JPQL (Java Persistence Query Language)	Native SQL Query
Đối tượng thao tác	Thao tác trên **Entity class & Java fields** (`Product`, `p.name`).	Thao tác trực tiếp trên **Database Tables & Columns** (`tbl_products`, `prod_name`).
Tính độc lập DB	Độc lập hoàn toàn. Chạy được trên cả PostgreSQL, MySQL, Oracle mà không cần sửa code.	Phụ thuộc vào cú pháp đặc thù của Database đang dùng (ví dụ: dùng hàm `JSONB` của Postgres).

Cú pháp	`@Query("SELECT p FROM Product p WHERE p.price > :minPrice")`	`@Query(value = "SELECT * FROM products WHERE price > :minPrice", nativeQuery = true)`

Code ví dụ Cập nhật dữ liệu hàng loạt với `@Modifying` :

Khi viết câu lệnh `UPDATE` hoặc `DELETE` thủ công bằng `@Query` , bắt buộc phải có `@Modifying` :

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

@Modifying(clearAutomatically = true) // clearAutomatically: Xóa sạch cache Persistence Contex

@Query("UPDATE Product p SET p.price = p.price * :rate WHERE p.category.id = :categoryId")

int increasePriceForCategory(@Param("categoryId") Long categoryId, @Param("rate") BigDecimal r

}
```

4.4 Phân trang (Pagination) & Sắp xếp (Sorting)

Khi bảng có hàng triệu bản ghi, bạn không bao giờ được phép `findAll()` về một `List` vì sẽ gây tràn RAM server (Out Of Memory). Luôn luôn dùng `Pageable` :

```
// Trong Service Layer:
int pageNumber = 0; // Trang đầu tiên (bắt đầu từ 0)

int pageSize = 20;  // Mỗi trang 20 phần tử

// Sắp xếp theo ngày tạo giảm dần (mới nhất lên đầu)

Pageable pageable = PageRequest.of(pageNumber, pageSize, Sort.by("createdAt").descending());

Page<Product> productPage = productRepository.findAll(pageable);

// Thông tin hữu ích từ Page object:

long totalItems = productPage.getTotalElements(); // Tổng số 5000 sản phẩm

int totalPages = productPage.getTotalPages();     // Tổng số 250 trang

List<Product> content = productPage.getContent(); // 20 sản phẩm của trang hiện tại

boolean isFirst = productPage.isFirst();

boolean isLast = productPage.isLast();
```

---

5. CÔNG NGHỆ DATA ACCESS KHÁC TRONG PLAYLIST

5.1 Spring Boot 3 `JdbcClient` (Truy vấn SQL tốc độ cao)

Trong Video 5 của chuỗi khóa học, khi bạn cần kiểm soát hoàn toàn 100% câu lệnh SQL mà không muốn overhead của Hibernate, Spring Boot 3 giới thiệu **`JdbcClient`** (Fluent API) thay thế cho `JdbcTemplate` cũ:

```
@Repository

@RequiredArgsConstructor


public class CustomProductJdbcDao {


private final JdbcClient jdbcClient;


public Optional<ProductResponse> findProductSummaryById(Long id) {


return jdbcClient.sql("""


SELECT p.id, p.name, p.price, c.name AS category_name, p.created_at


FROM products p


INNER JOIN categories c ON p.category_id = c.id


WHERE p.id = :id


""")


.param("id", id)


.query(ProductResponse.class) // Tự động map vào Java Record ProductResponse cực nhanh!


.optional();


}


}
```

5.2 NoSQL với Spring Data MongoDB (Video 4)

Khác với cơ sở dữ liệu quan hệ (SQL) lưu theo bảng và dòng, MongoDB lưu dữ liệu dưới dạng **Document (Tài liệu JSON/BSON linh hoạt)**.

Khai báo trong `pom.xml`:

```
<dependency>
<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-mongodb</artifactId>

</dependency>
```

Entity MongoDB (`@Document`):

```
package com.example.model;

import lombok.AllArgsConstructor;


import lombok.Data;


import lombok.NoArgsConstructor;


import org.bson.types.ObjectId;


import org.springframework.data.annotation.Id;


import org.springframework.data.mongodb.core.mapping.Document;


import org.springframework.data.mongodb.core.mapping.DocumentReference;


import java.util.List;


@Document(collection = "movies") // Ánh xạ tới collection 'movies' trong MongoDB


@Data


@AllArgsConstructor


@NoArgsConstructor


public class Movie {


@Id


private ObjectId id;


private String imdbId;
```

```
private String title;

private String releaseDate;

private String trailerLink;

private List<String> genres;

// Tham chiếu tới danh sách Review ở collection khác (tương tự Foreign Key trong SQL)

@DocumentReference

private List<Review> reviewIds;

}
```

Repository MongoDB:

```
package com.example.repository;

import com.example.model.Movie;


import org.bson.types.ObjectId;


import org.springframework.data.mongodb.repository.MongoRepository;


import org.springframework.stereotype.Repository;


import java.util.Optional;


@Repository


public interface MovieRepository extends MongoRepository<Movie, ObjectId> {


Optional<Movie> findMovieByImdbId(String imdbId);


}
```

Cú pháp của `MongoRepository` giống hệt `JpaRepository`, giúp bạn chuyển đổi mượt mà giữa SQL và NoSQL mà không phải học lại từ đầu!

CHƯƠNG 04: SERVICE LAYER, NGHIỆP VỤ & QUẢN LÝ TRANSACTION

(Business Logic Layer, Declarative Transactions & Async Processing)

---

1. THIẾT KẾ TẦNG DỊCH VỤ (SERVICE LAYER DESIGN)

1.1 Vai trò của Service Layer (What is it?)

- **Định nghĩa:** Service Layer (Tầng nghiệp vụ) là trung tâm đầu não của một ứng dụng backend. Tầng này chịu trách nhiệm:

  - Tiếp nhận các DTO từ Controller.

  - Phối hợp (Orchestrate) nhiều Repository, API bên thứ ba, hàng đợi tin nhắn (Message Queue).

  - Thực thi các quy tắc nghiệp vụ (Business Rules) và kiểm tra logic (ví dụ: số dư tài khoản có đủ không, hàng còn trong kho không).

  - Quản lý ranh giới giao dịch cơ sở dữ liệu (Transaction Boundary).

1.2 Tranh luận kiến trúc: Interface + Impl vs Concrete Class

Trong cộng đồng Spring, có 2 trường phái thiết kế Service:

1. **Trường phái Cổ điển (Interface + Implementation):**

```
OrderService (Interface) <—— OrderServiceImpl (@Service)
```

- *Khi nào nên dùng:* Khi dự án lớn có khả năng một Interface có **nhiều hơn một Implementation** (ví dụ: `PaymentService` có `VnpayServiceImpl` và `PaypalServiceImpl`), hoặc khi làm việc theo mô hình Outsourcing tách biệt hợp đồng API trước khi code.

2. **Trường phái Hiện đại (Concrete Class trực tiếp):**

```
OrderService (@Service)  // Không cần tạo interface rỗng chỉ có 1 class con
```

- *Khi nào nên dùng:* Trong các dự án vừa và nhỏ, microservices, khi 99% các service chỉ có duy nhất 1 class hiện thực. Giúp giảm 50% số lượng file và điều hướng code nhanh chóng. Từ Spring 3+ với CGLIB Proxy, Spring không còn bắt buộc phải có Interface để tạo Transaction Proxy nữa.

2. QUẢN LÝ GIAO DỊCH VỚI `@Transactional` (TRANSACTION MANAGEMENT)

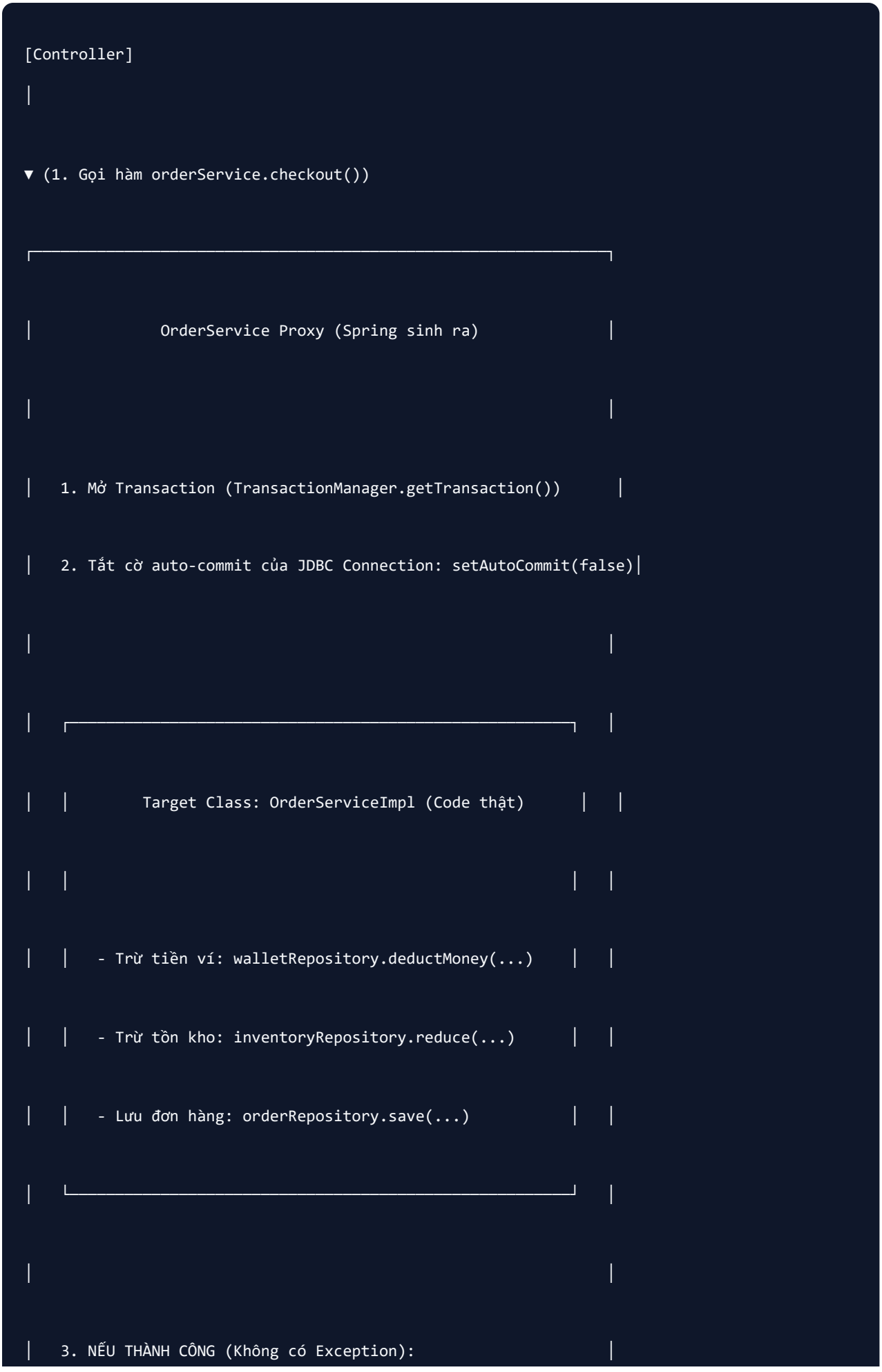
2.1 Bản chất của Transaction (ACID)

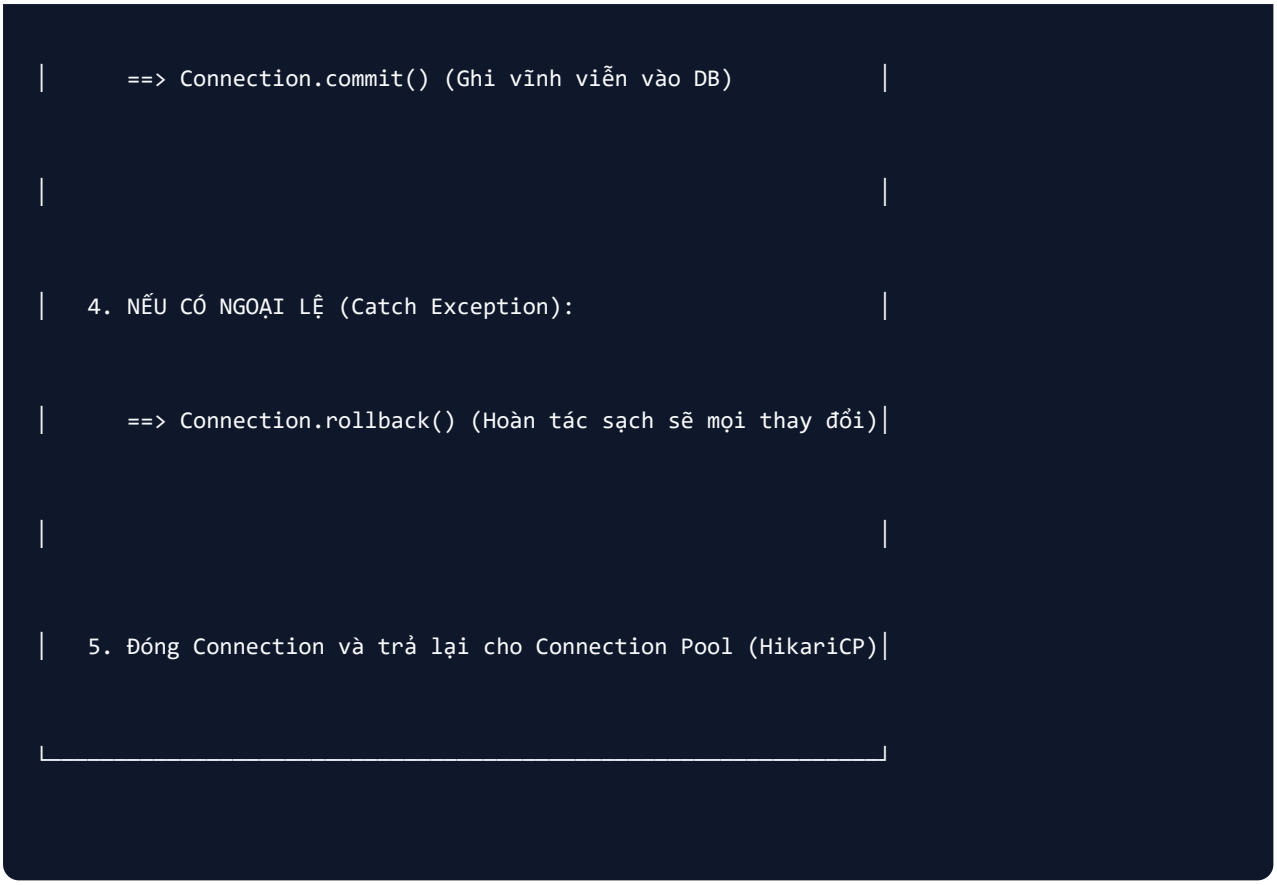
Một giao dịch cơ sở dữ liệu phải đảm bảo 4 tính chất vàng:

- **Atomicity (Nguyên tử):** "Tất cả hoặc không gì cả". Nếu bước A thành công nhưng bước B thất bại, toàn bộ thao tác của bước A phải được hoàn tác (Rollback) như chưa từng xảy ra.
- **Consistency (Nhất quán):** Dữ liệu luôn chuyển từ trạng thái hợp lệ này sang trạng thái hợp lệ khác.
- **Isolation (Cô lập):** Các giao dịch chạy đồng thời không được nhìn thấy dữ liệu trung gian chưa commit của nhau.
- **Durability (Bền vững):** Một khi giao dịch đã commit thành công, dữ liệu sẽ được lưu vĩnh viễn dù hệ thống có bị mất điện đột ngột.

2.2 Cơ chế hoạt động của `@Transactional` dưới góc nhìn AOP Proxy

Khi bạn đánh dấu `@Transactional` lên một phương thức, Spring **không can thiệp vào code gốc của bạn**, mà nó tự động tạo ra một lớp vỏ bọc ảo gọi là **Spring AOP Dynamic Proxy**:


[Controller]
▼ (1. Gọi hàm orderService.checkout())
OrderService Proxy (Spring sinh ra)
1. Mở Transaction (TransactionManager.getTransaction())
2. Tắt cờ auto-commit của JDBC Connection: setAutoCommit(false)
Target Class: OrderServiceImpl (Code thật)
- Trừ tiền ví: walletRepository.deductMoney(...)
- Trừ tồn kho: inventoryRepository.reduce(...)
- Lưu đơn hàng: orderRepository.save(...)
3. NẾU THÀNH CÔNG (Không có Exception):



2.3 CẠM BẪY SỐ 1: Quy tắc Rollback mặc định của Spring (Rollback Rules)

> 🚨 **CẢNH BÁO QUAN TRỌNG:**

> Mặc định trong Spring, `@Transactional` **CHỈ TỰ ĐỘNG ROLLBACK KHI GẶP UNCHECKED EXCEPTION** (`RuntimeException` và `Error`).

> Nếu code của bạn ném ra **CHECKED EXCEPTION** (như `IOException` , `SQLException` , hoặc class kế thừa từ `Exception`), **SPRING SẼ VẪN COMMIT TRANSACTION BÌNH THƯỜNG!**

Minh họa lỗi nguy hiểm:

```
// ❌ CỰC KỲ NGUY HIỂM:
@Transactional

public void transferMoney(Long fromId, Long toId, BigDecimal amount) throws Exception {

deduct(fromId, amount); // Trừ tiền thành công

// Gặp lỗi Checked Exception

if (networkFailed) {

throw new Exception("Lỗi mạng!");

// 🚨 HẬU QUẢ: Spring KHÔNG rollback! Tài khoản nguồn mất tiền, tài khoản đích không nhận được

}

add(toId, amount);

}
```

✅ **Cách khắc phục chuẩn mực:**

Luôn khai báo rõ ràng `rollbackFor = Exception.class`:

```
@Transactional(rollbackFor = Exception.class)

public void transferMoney(Long fromId, Long toId, BigDecimal amount) throws Exception {


// Bây giờ, BẤT KỲ Exception nào ném ra đều sẽ được Rollback 100%!


}
```

2.4 CẠM BẪY SỐ 2: Lỗi gọi nội bộ (The Self-Invocation Trap)

Đây là câu hỏi kiểm tra độ sâu kiến thức phổ biến nhất khi phỏng vấn Spring Boot Senior:

```
@Service
public class OrderService {

public void createOrder() {

// Gọi hàm nội bộ cùng class

this.processPayment();

}

@Transactional

public void processPayment() {

// Thao tác ghi Database

}

}
```

❓ **Câu hỏi:** Khi Controller gọi `orderService.createOrder()`, giao dịch trong hàm `processPayment()` có được kích hoạt không?

❗ **Đáp án: KHÔNG HỀ CÓ TRANSACTION NÀO ĐƯỢC MỞ!**

- **Nguyên nhân:** Vì lời gọi `this.processPayment()` là cuộc gọi nội bộ trực tiếp giữa các hàm trong cùng 1 đối tượng, nó **bỏ qua hoàn toàn lớp vỏ bọc Spring AOP Proxy**!

✅ **2 Cách khắc phục:**

1. Tách hàm `processPayment()` sang một Service riêng biệt (ví dụ: `PaymentService`) rồi inject vào.

2. Hoặc tự inject chính mình thông qua Context/ObjectProvider (Self-injection).

2.5 Các thuộc tính quan trọng của `@Transactional`

1. `readOnly = true` (Tối ưu hóa hiệu năng cực đại cho hàm đọc)

```
@Transactional(readOnly = true)
public ProductResponse getProductById(Long id) {

return productRepository.findById(id)...;

}
```

- **Lợi ích:**

1. Báo cho Hibernate biết không cần lưu snapshot của Entity vào Persistence Context ➔ **Tiết kiệm 50% bộ nhớ RAM**.

2. Bỏ qua hoàn toàn bước Dirty Checking khi kết thúc hàm ➔ **Tăng tốc độ truy vấn đáng kể**.

3. Với các hệ thống cơ sở dữ liệu có thiết lập Cluster (Master - Slave), Spring sẽ tự động định tuyến (route) câu truy vấn này sang máy chủ đọc (Read Replica) để giảm tải cho máy chủ ghi.

---

2. Propagation (Cơ chế lan truyền giao dịch)

Định nghĩa cách một hàm `@Transactional` hành xử khi nó được gọi từ bên trong một hàm `@Transactional` khác:

Propagation Type	Hành vi hoạt động	Trường hợp sử dụng thực tế
`REQUIRED` **(Mặc định)**	Nếu đã có Transaction cha đang chạy thì **dùng chung**. Nếu chưa có thì **tạo mới một Transaction**.	Dùng cho 95% các nghiệp vụ thông thường.
`REQUIRES_NEW`	**Luôn luôn tạo một Transaction độc lập hoàn toàn mới**. Nếu đang có Transaction cha, tạm dừng (suspend) Transaction cha lại, chạy xong Transaction mới này thì quay lại tiếp tục cha.	**Ghi Log kiểm toán (Audit Log) hoặc Lưu lịch sử giao dịch.** Dù nghiệp vụ chính có bị lỗi rollback thì bản ghi log vẫn phải được commit lưu lại!
`SUPPORTS`	Nếu có thì chạy trong transaction, không có thì chạy không cần transaction.	Các hàm tìm kiếm, đọc dữ liệu linh hoạt.

`MANDATORY`	Bắt buộc phải có Transaction đang chạy từ trước. Nếu gọi độc lập sẽ ném lỗi `IllegalTransactionStateException`.	Dùng khi muốn bảo vệ hàm con, không cho phép ai gọi lẻ tẻ bên ngoài.
`NEVER`	Cấm tuyệt đối không được có transaction. Ném lỗi nếu phát hiện có transaction.	Thao tác không an toàn với transaction.

Code ví dụ thực tế sử dụng `REQUIRES_NEW`:

```
@Service

@RequiredArgsConstructor


public class CheckoutService {


private final OrderRepository orderRepository;


private final AuditLogService auditLogService;


@Transactional(rollbackFor = Exception.class)


public void checkout(OrderRequest request) {


try {


// Bước 1: Ghi log bắt đầu thanh toán


auditLogService.logAttempt(request.getUserId(), "Bắt đầu thanh toán");


// Bước 2: Trừ tiền, tạo đơn (bị lỗi ném RuntimeException)


processOrder(request);


} catch (Exception e) {


// Bước 3: Ghi log thất bại


auditLogService.logAttempt(request.getUserId(), "Thanh toán thất bại: " + e.getMessage());


throw e; // Ném lỗi để rollback bước tạo đơn


}
```

```
}

}

@Service

@RequiredArgsConstructor

public class AuditLogService {

private final AuditLogRepository auditLogRepository;

// BẮT BUỘC: REQUIRES_NEW để việc lưu log diễn ra trong một giao dịch riêng,

// không bao giờ bị rollback theo đơn hàng!

@Transactional(propagation = Propagation.REQUIRES_NEW)

public void logAttempt(Long userId, String action) {

auditLogRepository.save(new AuditLog(userId, action, Instant.now()));

}

}
```

---

3. XỬ LÝ BẤT ĐỒNG BỘ ĐA LUỒNG VỚI `@Async`

3.1 Bài toán thực tế

Trong Video 6 (Xây dựng Reddit Clone), khi người dùng bấm nút Đăng ký tài khoản (`User Registration`), hệ thống cần phải gửi một bức email xác thực tài khoản kèm mã kích hoạt.

- Việc kết nối tới Mail Server (SMTP) để gửi thư thường mất từ **2 đến 5 giây**.
- Nếu xử lý đồng bộ (Synchronous), người dùng sẽ phải nhìn màn hình quay vòng vòng chờ đợi suốt 5 giây.
- 👉 **Giải pháp:** Phản hồi ngay lập tức `201 Created: "Đăng ký thành công, vui lòng kiểm tra hộp thư!"` về cho người dùng (chỉ mất 50ms), đồng thời đẩy tác vụ gửi email cho một luồng chạy ngầm (Background Thread / Worker) xử lý riêng biệt.

3.2 Cấu hình Thread Pool chuẩn cho `@Async`

1. Tạo file cấu hình `AsyncConfig.java`:

```
package com.example.config;

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;

import org.springframework.scheduling.annotation.EnableAsync;

import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

import java.util.concurrent.Executor;

@Configuration

@EnableAsync // Kích hoạt tính năng xử lý bất đồng bộ

public class AsyncConfig {

@Bean(name = "mailTaskExecutor")

public Executor mailTaskExecutor() {

ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();

executor.setCorePoolSize(5);        // Luôn duy trì tối thiểu 5 luồng sẵn sàng

executor.setMaxPoolSize(20);        // Có thể mở rộng tối đa lên 20 luồng khi quá tải

executor.setQueueCapacity(500);     // Hàng đợi chứa tối đa 500 email chờ gửi

executor.setThreadNamePrefix("MailWorker-"); // Đặt tên luồng để dễ dàng debug trong log
```

```
executor.initialize();

return executor;

}

}
```

2. Viết Service gửi thư có `@Async`:

```
package com.example.service;

import lombok.RequiredArgsConstructor;

import lombok.extern.slf4j.Slf4j;

import org.springframework.mail.javamail.JavaMailSender;

import org.springframework.mail.javamail.MimeMessageHelper;

import org.springframework.scheduling.annotation.Async;

import org.springframework.stereotype.Service;

@Service

@RequiredArgsConstructor

@Slf4j

public class MailService {

private final JavaMailSender mailSender;

// Chỉ định rõ thực thi trên thread pool 'mailTaskExecutor'

@Async("mailTaskExecutor")

public void sendAccountActivationEmail(String recipientEmail, String activationToken) {

log.info("Bắt đầu gửi email ngầm trên luồng: {}", Thread.currentThread().getName());

try {
```

```
// Giả lập thời gian gửi email qua SMTP

Thread.sleep(3000);

// Code gửi mail thực tế...

log.info("Email kích hoạt đã gửi thành công tới: {}", recipientEmail);

} catch (InterruptedException e) {

log.error("Lỗi gián đoạn khi gửi email", e);

Thread.currentThread().interrupt();

}

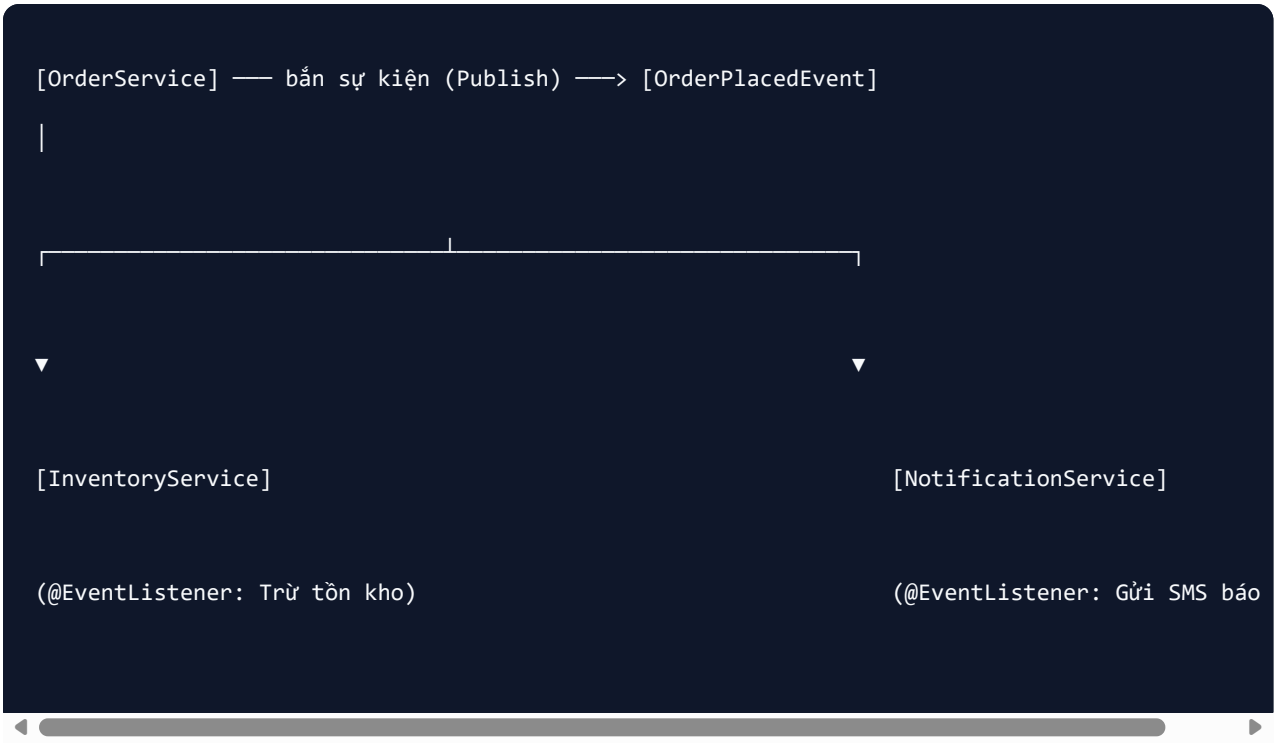
}

}
```

---

4. BẮN VÀ LẮNG NGHE SỰ KIỆN NỘI BỘ (APPLICATION EVENTS)

Để code đạt chuẩn **Loose Coupling (Tách rời phụ thuộc)** cao nhất, thay vì Service A gọi trực tiếp Service B, ta sử dụng mô hình **Event-Driven Architecture** tích hợp sẵn trong Spring.



4.1 Định nghĩa Event (Dùng Java Record):

```
package com.example.event;

public record OrderPlacedEvent(Long orderId, String customerEmail, Double totalAmount) {}
```

4.2 Bắn sự kiện từ Service:

```
@Service

@RequiredArgsConstructor


public class OrderService {


private final ApplicationEventPublisher eventPublisher; // Spring tự inject


@Transactional


public void placeOrder(OrderRequest request) {


// Lưu đơn hàng vào Database...


Long newOrderId = 999L;


// Bắn sự kiện ra ngoài toàn hệ thống


eventPublisher.publishEvent(new OrderPlacedEvent(newOrderId, request.getEmail(), 500.0));


}


}
```

4.3 Lắng nghe sự kiện (`@EventListener`):

```
@Component

@Slf4j


public class NotificationEventListener {


// Đặc biệt: @TransactionalEventListener(phase = AFTER_COMMIT)


// Chỉ gửi thông báo KHI VÀ CHỈ KHI giao dịch lưu đơn hàng ở trên đã COMMIT THÀNH CÔNG vào DB!


// Nếu lưu đơn bị lỗi rollback thì tuyệt đối không gửi tin nhắn spam khách hàng.


@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)


public void handleOrderPlaced(OrderPlacedEvent event) {


log.info("Giao dịch thành công! Gửi SMS chúc mừng tới khách hàng: {}", event.customerEmail());


}


}
```

Nhờ mô hình này, `OrderService` hoàn toàn không cần biết sự tồn tại của `NotificationService` . Khi muốn thêm tính năng mới (ví dụ: Tích điểm thành viên), bạn chỉ cần tạo thêm 1 Listener mới mà không cần sửa 1 dòng code nào trong `OrderService` !

CHƯƠNG 05: TÍNH NĂNG HIỆN ĐẠI CỦA SPRING BOOT 3 & HTTP CLIENTS

(Modern Spring Boot 3, RestClient, OpenAPI/Swagger & Actuator)

---

1. CUỘC CÁCH MẠNG SPRING BOOT 3 & JAVA 17/21 BASELINE

Spring Boot 3 (ra mắt từ cuối năm 2022 và phát triển mạnh mẽ trên phiên bản 3.2+) đánh dấu bước chuyển mình lớn nhất trong lịch sử 10 năm của Spring Boot.

1.1 Những thay đổi bắt buộc phải biết (Breaking Changes)

1. Yêu cầu tối thiểu Java 17 (Khuyến nghị Java 21 LTS):

Spring Boot 3 không còn hỗ trợ Java 8 hay Java 11. Ứng dụng tận dụng tối đa các tính năng mới của Java hiện đại:

- **Records:** Dùng làm DTO bất biến, cú pháp cực ngắn gọn.
- **Pattern Matching cho `switch`:** Xử lý các điều kiện rẽ nhánh sạch sẽ, an toàn kiểu dữ liệu.
- **Text Blocks (`"""`):** Viết câu lệnh SQL nhiều dòng hoặc JSON payload trong code mà không cần nối chuỗi `+ "\n"`.
- **Virtual Threads (Project Loom - Java 21):** Chỉ cần bật 1 dòng cấu hình:

```
  spring:
threads:


virtual:


enabled: true
```

Ứng dụng có thể xử lý hàng chục nghìn request đồng thời mà không bị nghẽn luồng (Thread starvation), với chi phí bộ nhớ cực kỳ thấp!

2. Di chuyển toàn bộ từ `javax.` *sang* `jakarta.` (Jakarta EE 10 Baseline):

Do tranh chấp thương hiệu giữa Oracle và Eclipse Foundation, toàn bộ các thư viện Java EE cũ đã đổi tên namespace:

- `javax.persistence.` ➔ *`jakarta.persistence.`* (JPA Entity, Id, Column)
- `javax.validation.` ➔ *`jakarta.validation.`* (Validation @NotNull, @Email)
- `javax.servlet.` ➔ *`jakarta.servlet.`* (HttpServletRequest, Filter)

3. Hỗ trợ GraalVM Native Image:

Biên dịch trước thời hạn (AOT - Ahead-Of-Time) biến ứng dụng Spring Boot thành một file thực thi nhị phân (Executable Binary) độc lập:

- Khởi động chỉ mất **0.05 giây (50 milliseconds)** thay vì 3-5 giây trên JVM.
- Sử dụng RAM chỉ từ **30MB - 50MB** thay vì 300MB - 500MB.
- Cực kỳ lý tưởng cho môi trường Serverless (AWS Lambda, Google Cloud Run) và kiến trúc Microservices trên Kubernetes.

---

2. GỌI API BÊN THỨ BA VỚI `RestClient` MỚI (SPRING BOOT 3.2+)

Trong các dự án thực tế, bạn liên tục phải gọi các API bên ngoài (ví dụ: Cổng thanh toán VNPay/Stripe, Hệ thống gửi SMS, Dịch vụ thời tiết, API ngân hàng...).

2.1 Lịch sử tiến hóa của HTTP Clients trong Spring

```
1. RestTemplate (2009)          --> Cổ điển, đồng bộ, nhưng thiết kế quá tải nhiều hàm overloa
2. WebClient (2017)             --> Hiện đại, Reactive (Non-blocking), nhưng bắt buộc phải kéo


3. RestClient (Spring Boot 3.2) --> ✨ "Chân ái" mới: Kết hợp cú pháp Fluent API hiện đại của
```

2.2 Cách sử dụng `RestClient` chi tiết

1. Đăng ký Bean `RestClient` trong cấu hình:

```
package com.example.config;

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;

import org.springframework.http.HttpHeaders;

import org.springframework.http.MediaType;

import org.springframework.web.client.RestClient;

@Configuration

public class RestClientConfig {

@Bean

public RestClient externalApiRestClient() {

return RestClient.builder()

.baseUrl("https://jsonplaceholder.typicode.com")

.defaultHeader(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)

.defaultHeader("X-App-Client", "SpringBoot3-Backend")

.build();

}
```

```
}
```

2. Thực hiện các thao tác HTTP GET, POST, PUT, DELETE:

```
package com.example.client;

import lombok.RequiredArgsConstructor;


import org.springframework.core.ParameterizedTypeReference;


import org.springframework.http.HttpStatusCode;


import org.springframework.stereotype.Service;


import org.springframework.web.client.RestClient;


import java.util.List;


public record PostDto(Long id, Long userId, String title, String body) {}


@Service


@RequiredArgsConstructor


public class ExternalPostService {


private final RestClient restClient;


// 1. GET: Lấy chi tiết 1 bài viết theo ID


public PostDto getPostById(Long id) {


return restClient.get()


.uri("/posts/{id}", id)


.retrieve()
```

```
// Bắt lỗi 4xx từ API bên ngoài

.onStatus(HttpStatusCode::is4xxClientError, (request, response) -> {

throw new RuntimeException("API đối tác báo không tìm thấy bài viết với mã ID: " + id);

})

// Bắt lỗi 5xx từ API bên ngoài

.onStatus(HttpStatusCode::is5xxServerError, (request, response) -> {

throw new RuntimeException("Máy chủ đối tác đang bị sập!");

})

.body(PostDto.class); // Tự động map JSON thành Java Record

}

// 2. GET: Lấy danh sách bài viết (List<PostDto>)

public List<PostDto> getAllPosts() {

return restClient.get()

.uri("/posts")

.retrieve()

// Dùng ParameterizedTypeReference để map chính xác Generic List

.body(new ParameterizedTypeReference<List<PostDto>>() {});
```

```
}

// 3. POST: Gửi JSON tạo mới bài viết

public PostDto createNewPost(PostDto newPost) {

return restClient.post()

.uri("/posts")

.body(newPost) // Jackson tự động serialize object thành JSON body

.retrieve()

.body(PostDto.class);

}

// 4. DELETE: Xóa bài viết

public void deletePost(Long id) {

restClient.delete()

.uri("/posts/{id}", id)

.retrieve()

.toBodilessEntity(); // Chỉ cần kiểm tra status thành công, không cần body

}

}
```

2.3 HTTP Interfaces Declarative (`@HttpExchange`) - Đỉnh cao lập trình Client

Tương tự như cách Spring Data JPA cho phép bạn chỉ cần khai báo Interface là có sẵn hàm truy vấn DB, Spring Boot 3 cho phép bạn **chỉ cần khai báo Interface là có sẵn HTTP Client gọi API ngoài**!

Bước 1: Định nghĩa Interface với các Annotation HTTP:

```
package com.example.client;

import org.springframework.web.bind.annotation.PathVariable;


import org.springframework.web.bind.annotation.RequestBody;


import org.springframework.web.service.annotation.*;


import java.util.List;


@HttpExchange("/posts")


public interface PostClient {


@GetExchange


List<PostDto> findAll();


@GetExchange("/{id}")


PostDto findById(@PathVariable("id") Long id);


@PostExchange


PostDto create(@RequestBody PostDto newPost);


@DeleteExchange("/{id}")


void delete(@PathVariable("id") Long id);


}
```

Bước 2: Đăng ký Proxy sinh code tự động:

```
package com.example.config;

import com.example.client.PostClient;


import org.springframework.context.annotation.Bean;


import org.springframework.context.annotation.Configuration;


import org.springframework.web.client.RestClient;


import org.springframework.web.client.support.RestClientAdapter;


import org.springframework.web.service.invoker.HttpServiceProxyFactory;


@Configuration


public class HttpInterfacesConfig {


@Bean


public PostClient postClient() {


RestClient restClient = RestClient.builder()


.baseUrl("https://jsonplaceholder.typicode.com")


.build();


// Sử dụng HttpServiceProxyFactory để tự động tạo Implementation cho PostClient


HttpServiceProxyFactory factory = HttpServiceProxyFactory


.builderFor(RestClientAdapter.create(restClient))
```

```
.build();

return factory.createClient(PostClient.class);

}

}
```

Giờ đây trong Service, bạn chỉ cần inject `PostClient postClient` và gọi `postClient.findById(1L)` như một hàm Java bình thường, Spring sẽ tự động gửi HTTP Request ngầm bên dưới!

---

3. TẠO TÀI LIỆU API TỰ ĐỘNG VỚI OPENAPI 3 & SWAGGER UI

Không ai còn viết tài liệu API bằng file Word hay Excel thủ công. Trong Spring Boot 3, ta sử dụng **SpringDoc OpenAPI 3** để hệ thống tự động quét code và tạo ra giao diện web tương tác trực quan.

3.1 Khai báo dependency trong `pom.xml` :

```
<dependency>

<groupId>org.springdoc</groupId>

<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>

<version>2.5.0</version>

</dependency>
```

3.2 Cấu hình thông tin API chung:

```
package com.example.config;

import io.swagger.v3.oas.models.Components;


import io.swagger.v3.oas.models.OpenAPI;


import io.swagger.v3.oas.models.info.Contact;


import io.swagger.v3.oas.models.info.Info;


import io.swagger.v3.oas.models.info.License;


import io.swagger.v3.oas.models.security.SecurityRequirement;


import io.swagger.v3.oas.models.security.SecurityScheme;


import org.springframework.context.annotation.Bean;


import org.springframework.context.annotation.Configuration;


@Configuration


public class OpenApiConfig {


@Bean


public OpenAPI customOpenAPI() {


return new OpenAPI()


.info(new Info()


.title("E-Commerce Backend REST API")
```

```
.version("1.0.0")

.description("Tài liệu kỹ thuật và thử nghiệm API cho ứng dụng bán hàng trực tuyến.")

.contact(new Contact().name("Đội ngũ Kỹ thuật").email("dev@mycompany.com"))

.license(new License().name("Apache 2.0")))

// Tích hợp nút nhập Bearer JWT Token trực tiếp trên giao diện Swagger UI

.addSecurityItem(new SecurityRequirement().addList("BearerAuth"))

.components(new Components()

.addSecuritySchemes("BearerAuth", new SecurityScheme()

.name("BearerAuth")

.type(SecurityScheme.Type.HTTP)

.scheme("bearer")

.bearerFormat("JWT")));

}

}
```

3.3 Đánh chú thích trực tiếp trên Controller:

```
@Tag(name = "Quản lý Sản phẩm (Product API)", description = "Các API phục vụ truy vấn và cập n

@RestController

@RequestMapping("/api/v1/products")

public class ProductController {

@Operation(

summary = "Lấy chi tiết sản phẩm theo ID",

description = "Trả về thông tin đầy đủ gồm giá tiền, tồn kho, danh mục của sản phẩm"

)

@ApiResponses({

@ApiResponse(responseCode = "200", description = "Tìm thấy sản phẩm thành công"),

@ApiResponse(responseCode = "404", description = "Không tìm thấy sản phẩm theo mã ID đã truyền

})

@GetMapping("/{id}")

public ResponseEntity<ProductResponse> getProductById(@PathVariable Long id) {

// ...

}
```

```
}
```

Truy cập giao diện Swagger:

Mở trình duyệt gõ: `http://localhost:8080/swagger-ui.html`

Bạn có thể xem toàn bộ các endpoint, cấu trúc JSON Request/Response, và bấm nút **"Try it out"** để test trực tiếp như Postman!

---

4. GIÁM SÁT HỆ THỐNG VỚI SPRING BOOT ACTUATOR

4.1 Actuator là gì? (What is it?)

- Trong môi trường Production, bạn cần biết máy chủ có đang sống khỏe không, còn bao nhiêu dung lượng đĩa cứng, bộ nhớ RAM JVM đang bị chiếm bao nhiêu %, và kết nối Database có bị gián đoạn hay không.
- **Spring Boot Actuator** cung cấp sẵn các HTTP Endpoint tích hợp sẵn phục vụ cho việc giám sát (Monitoring) và kiểm tra tình trạng ứng dụng (Health Checks).

4.2 Cài đặt & Cấu hình trong `application.yml`

```
<dependency>
<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-actuator</artifactId>

</dependency>
```

```
management:

endpoints:


web:


exposure:


# Chỉ mở các endpoint an toàn ra ngoài, không mở toàn bộ vì lý do bảo mật


include: health, info, metrics


endpoint:


health:


show-details: always # Hiển thị chi tiết tình trạng Database, Disk space
```

4.3 Các Endpoint Actuator quan trọng nhất:

Endpoint URL	Mục đích sử dụng thực tế
`/actuator/health`	**Health Check:** Kiểm tra xem ứng dụng và cơ sở dữ liệu có đang hoạt động tốt hay không (`"status": "UP"`). Endpoint này bắt buộc phải có để các hệ thống điều phối như **Kubernetes**, **Docker Swarm**, hay **AWS ALB (Application Load Balancer)** biết khi nào server bị chết để tự động khởi động lại.
`/actuator/metrics`	Đo lường hiệu năng: Số lượng HTTP Request theo giây, thời gian phản hồi trung bình, số luồng đang chạy (`jvm.threads.live`), bộ nhớ RAM JVM Heap (`jvm.memory.used`). Tích hợp sẵn với hệ thống Prometheus & Grafana để vẽ biểu đồ giám sát trực quan thời gian thực.
`/actuator/info`	Hiển thị thông tin phiên bản git commit, ngày build của ứng dụng.

CHƯƠNG 06: BẢO MẬT CHUYÊN SÂU: SPRING SECURITY & JWT

(Production Spring Security 6, JWT Authentication, Refresh Tokens & RBAC)

---

1. TỔNG QUAN KIẾN TRÚC SPRING SECURITY 6 TRÊN SPRING BOOT 3

Trong Spring Boot 3, lớp cấu hình cũ `WebSecurityConfigurerAdapter` đã bị **xóa bỏ hoàn toàn**. Thay vào đó, Spring Security chuyển sang kiến trúc hoàn toàn hướng Component dựa trên việc đăng ký Bean `SecurityFilterChain` với cú pháp Lambda DSL hiện đại.

1.1 Cơ chế chuỗi bộ lọc (Filter Chain Architecture)

Mọi HTTP Request từ bên ngoài gửi tới ứng dụng Spring Boot đều phải đi qua một chuỗi các bộ lọc bảo vệ trước khi tới được Controller:


[HTTP Request từ Client: Bearer JWT...]
Spring Security Filter Chain
1. CorsFilter (Kiểm tra nguồn gốc Domain gọi API)
2. CsrfFilter (Bảo vệ chống tấn công CSRF)
3. JwtAuthenticationFilter (⭐ BỘ LỌC TỰ VIẾT)
- Trích xuất Header "Authorization: Bearer <token>"
- Giải mã chữ ký số & kiểm tra hạn sử dụng của JWT
- Nạp thông tin User vào SecurityContextHolder
4. UsernamePasswordAuthenticationFilter
5. AuthorizationFilter (Kiểm tra quyền Role ADMIN / USER)
(Nếu xác thực & phân quyền thành công)

```
[Tới REST Controller]
```

1.2 Các thành phần cốt lõi trong Spring Security

1. **`SecurityContextHolder`:** Nơi lưu trữ thông tin về người dùng đang đăng nhập trong luồng thực thi hiện tại (ThreadLocal).

2. **`Authentication`:** Đối tượng đại diện cho phiên làm việc hiện tại, chứa:

- `Principal`: Đối tượng người dùng (thường là `UserDetails`).

- `Credentials`: Mật khẩu (đã xóa sau khi xác thực thành công).

- `Authorities`: Danh sách các quyền hạn hoặc vai trò (`ROLE_USER`, `ROLE_ADMIN`).

3. **`AuthenticationManager`:** Nhận request đăng nhập (username/password), điều phối các `AuthenticationProvider` để kiểm tra thông tin.

4. **`UserDetailsService`:** Interface cầu nối kết nối Spring Security với Database của bạn để tìm user theo username/email.

5. **`PasswordEncoder`:** Thư viện băm mật khẩu một chiều chống giải mã (sử dụng thuật toán **BCrypt**).

---

2. CƠ CHẾ XÁC THỰC DỰA TRÊN TOKEN (JWT - JSON WEB TOKEN)

2.1 Tại sao REST API lại dùng JWT thay vì Session Cookie truyền thống?

Tiêu chí	Session Cookie truyền thống	Token-based Authentication (JWT)
Nơi lưu trữ	Lưu trữ trên **RAM của Server** (Stateful).	Lưu trữ phía **Client** (LocalStorage, Cookie, Mobile SecureStorage) (Stateless).
Khả năng mở rộng (Scalability)	Kém. Khi có 10 server chạy song song, bạn phải cấu hình Sticky Session hoặc Redis Session dùng chung phức tạp.	Tuyệt đối. Server chỉ cần kiểm tra chữ ký số mật, không cần lưu bất cứ gì trong RAM server. Bất kỳ server nào cũng xác thực được.
Hỗ trợ Đa nền tảng	Khó tích hợp với Mobile App (iOS/Android), IoT, ứng dụng Single Page (React/Angular).	Tương thích 100% với mọi nền tảng (Web, Mobile, Desktop app).

2.2 Cấu trúc giải phẫu một chuỗi JWT (Anatomy of JWT)

Một chuỗi JWT gồm 3 phần được nối với nhau bởi 2 dấu chấm (`.`): `header.payload.signature`

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c2VyQGV4YW1wbGUuY29tIiwicm9sZXMiOlsiUk9MRV9VU
```

1. **Header (Đầu):** Chứa thuật toán mã hóa (VD: `HS256` hoặc `RS256`) và loại token (`JWT`).

2. **Payload (Thân):** Chứa các thông tin định danh (Claims):

- Registered Claims: `sub` (Subject - username/email), `iat` (Issued At - thời điểm tạo), `exp` (Expiration - thời điểm hết hạn).

- Custom Claims: `roles: ["ROLE_ADMIN"]` , `userId: 101` .

- ⚠️ *Lưu ý:* Payload chỉ được mã hóa dạng Base64Url (ai cũng đọc được nội dung), vì vậy **tuyệt đối không bao giờ để mật khẩu hay số thẻ tín dụng vào Payload của JWT**!

3. **Signature (Chữ ký số):** Được tính toán bằng công thức:

$$\text{Signature} = \text{HMAC-SHA256}(\text{Base64Url}(\text{Header}) + "." + \text{Base64Url}(\text{Payload}), \text{SecretKey})$$

Nếu hacker cố tình sửa đổi `role` từ `ROLE_USER` thành `ROLE_ADMIN` trong Payload, chữ ký số sẽ không còn khớp với SecretKey của Server ➔ Token lập tức bị từ chối!

---

3. HIỆN THỰC HÓA BỘ XÁC THỰC JWT HOÀN CHỈNH TỪ CON SỐ 0

3.1 Cài đặt thư viện JJWT trong `pom.xml`:

```
<!-- Thư viện tạo và xác thực JWT chuẩn thế giới -->

<dependency>

<groupId>io.jsonwebtoken</groupId>

<artifactId>jjwt-api</artifactId>

<version>0.12.5</version>

</dependency>

<dependency>

<groupId>io.jsonwebtoken</groupId>

<artifactId>jjwt-impl</artifactId>

<version>0.12.5</version>

<scope>runtime</scope>

</dependency>

<dependency>

<groupId>io.jsonwebtoken</groupId>

<artifactId>jjwt-jackson</artifactId>

<version>0.12.5</version>
```

```
<scope>runtime</scope>


</dependency>
```

3.2 Lớp tiện ích `JwtService.java` (Sinh và Giải mã Token)

```
package com.example.security;

import io.jsonwebtoken.Claims;

import io.jsonwebtoken.Jwts;

import io.jsonwebtoken.io.Decoders;

import io.jsonwebtoken.security.Keys;

import org.springframework.beans.factory.annotation.Value;

import org.springframework.security.core.userdetails.UserDetails;

import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;

import java.util.Date;

import java.util.HashMap;

import java.util.Map;

import java.util.function.Function;

@Service

public class JwtService {

// Chuỗi bí mật 256-bit định dạng Base64 cấu hình trong application.yml

@Value("${app.jwt.secret}")
```

```
private String secretKey;

@Value("${app.jwt.access-token-expiration-ms:900000}") // Mặc định: 15 phút (900.000 ms)

private long jwtExpirationMs;

// 1. Trích xuất Username (Subject) từ Token

public String extractUsername(String token) {

return extractClaim(token, Claims::getSubject);

}

public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {

final Claims claims = extractAllClaims(token);

return claimsResolver.apply(claims);

}

// 2. Sinh Access Token cho người dùng

public String generateToken(UserDetails userDetails) {

return generateToken(new HashMap<>(), userDetails);

}

public String generateToken(Map<String, Object> extraClaims, UserDetails userDetails) {

return Jwts.builder()
```

```
.claims(extraClaims)

.subject(userDetails.getUsername())

.issuedAt(new Date(System.currentTimeMillis()))

.expiration(new Date(System.currentTimeMillis() + jwtExpirationMs))

.signWith(getSignInKey(), Jwts.SIG.HS256) // Ký bằng thuật toán HMAC-SHA256

.compact();

}

// 3. Kiểm tra tính hợp lệ của Token

public boolean isTokenValid(String token, UserDetails userDetails) {

final String username = extractUsername(token);

return (username.equals(userDetails.getUsername())) && !isTokenExpired(token);

}

private boolean isTokenExpired(String token) {

return extractExpiration(token).before(new Date());

}

private Date extractExpiration(String token) {

return extractClaim(token, Claims::getExpiration);
```

```
}

private Claims extractAllClaims(String token) {

return Jwts.parser()

.verifyWith(getSignInKey())

.build()

.parseSignedClaims(token)

.getPayload();

}

private SecretKey getSignInKey() {

byte[] keyBytes = Decoders.BASE64.decode(secretKey);

return Keys.hmacShaKeyFor(keyBytes);

}

}
```

3.3 Bộ lọc đánh chặn `JwtAuthenticationFilter.java`

Bộ lọc này kế thừa từ `OncePerRequestFilter`, đảm bảo nó chỉ được thực thi **đúng một lần duy nhất cho mỗi HTTP Request**:

```
package com.example.security;

import jakarta.servlet.FilterChain;


import jakarta.servlet.ServletException;


import jakarta.servlet.http.HttpServletRequest;


import jakarta.servlet.http.HttpServletResponse;


import lombok.RequiredArgsConstructor;


import org.springframework.lang.NonNull;


import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;


import org.springframework.security.core.context.SecurityContextHolder;


import org.springframework.security.core.userdetails.UserDetails;


import org.springframework.security.core.userdetails.UserDetailsService;


import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;


import org.springframework.stereotype.Component;


import org.springframework.web.filter.OncePerRequestFilter;


import java.io.IOException;


@Component


@RequiredArgsConstructor


public class JwtAuthenticationFilter extends OncePerRequestFilter {
```

```
private final JwtService jwtService;

private final UserDetailsService userDetailsService;

@Override

protected void doFilterInternal(

@NonNull HttpServletRequest request,

@NonNull HttpServletResponse response,

@NonNull FilterChain filterChain

) throws ServletException, IOException {

// Bước 1: Lấy Header "Authorization" từ Request

final String authHeader = request.getHeader("Authorization");

final String jwt;

final String userEmail;

// Nếu không có header hoặc header không bắt đầu bằng "Bearer ", bỏ qua chuyển sang filter tiế

if (authHeader == null                !authHeader.startsWith("Bearer ")) {

filterChain.doFilter(request, response);

return;

}
```

```
// Bước 2: Cắt bỏ 7 ký tự đầu ("Bearer ") để lấy ra chuỗi Token nguyên gốc

jwt = authHeader.substring(7);

try {

userEmail = jwtService.extractUsername(jwt);

// Bước 3: Nếu có email và trong SecurityContextHolder chưa có thông tin xác thực

if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {

UserDetails userDetails = this.userDetailsService.loadUserByUsername(userEmail);

// Bước 4: Kiểm tra token có hợp lệ không

if (jwtService.isTokenValid(jwt, userDetails)) {

UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(

userDetails,

null,

userDetails.getAuthorities() // Nạp danh sách quyền (Roles)

);

authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

// ⭐ BƯỚC QUYẾT ĐỊNH: Nạp Authentication vào Context để Spring Security biết người này đã đăn

SecurityContextHolder.getContext().setAuthentication(authToken);
```

```
}

}

} catch (Exception e) {

// Token không hợp lệ hoặc hết hạn: bỏ qua không set authentication

}

// Cho phép request đi tiếp tới Controller

filterChain.doFilter(request, response);

}

}
```

3.4 Cấu hình `SecurityFilterChain` & Mở cổng API (`SecurityConfig.java`)

```
package com.example.config;

import com.example.security.JwtAuthenticationFilter;


import lombok.RequiredArgsConstructor;


import org.springframework.context.annotation.Bean;


import org.springframework.context.annotation.Configuration;


import org.springframework.security.authentication.AuthenticationManager;


import org.springframework.security.authentication.AuthenticationProvider;


import org.springframework.security.authentication.dao.DaoAuthenticationProvider;


import org.springframework.security.config.annotation.authentication.configuration.Authenticat


import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurit


import org.springframework.security.config.annotation.web.builders.HttpSecurity;


import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;


import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;


import org.springframework.security.config.http.SessionCreationPolicy;


import org.springframework.security.core.userdetails.UserDetailsService;


import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;


import org.springframework.security.crypto.password.PasswordEncoder;
```

```
import org.springframework.security.web.SecurityFilterChain;

import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

import org.springframework.web.cors.CorsConfiguration;

import org.springframework.web.cors.CorsConfigurationSource;

import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@Configuration

@EnableWebSecurity

@EnableMethodSecurity // Kích hoạt phân quyền theo method với @PreAuthorize

@RequiredArgsConstructor

public class SecurityConfig {

private final JwtAuthenticationFilter jwtAuthFilter;

private final UserDetailsService userDetailsService;

@Bean

public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

http

// 1. Tắt CSRF vì REST API dùng JWT là Stateless, không dùng Session Cookie nên không bị tấn c
```

```
.csrf(AbstractHttpConfigurer::disable)

// 2. Bật CORS để cho phép Frontend (React, Angular) gọi API

.cors(cors -> cors.configurationSource(corsConfigurationSource()))

// 3. Phân quyền Endpoint

.authorizeHttpRequests(auth -> auth

// Các URL công khai ai cũng vào được

.requestMatchers(

"/api/v1/auth/",

"/swagger-ui/",

"/v3/api-docs/",

"/actuator/health"

).permitAll()

// Các URL chỉ dành cho ADMIN

.requestMatchers("/api/v1/admin/").hasRole("ADMIN")

// Mọi request còn lại bắt buộc phải đăng nhập

.anyRequest().authenticated()

)
```

```
// 4. Cấu hình Session Policy là STATELESS (Không lưu Session phía server)

.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))

// 5. Khai báo Authentication Provider

.authenticationProvider(authenticationProvider())

// 6. Gắn JwtAuthenticationFilter chạy TRƯỚC UsernamePasswordAuthenticationFilter

.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

return http.build();

}

@Bean

public AuthenticationProvider authenticationProvider() {

DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();

authProvider.setUserDetailsService(userDetailsService);

authProvider.setPasswordEncoder(passwordEncoder());

return authProvider;

}

@Bean

public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
```

```
return config.getAuthenticationManager();

}

// Bean mã hóa mật khẩu bằng BCrypt

@Bean

public PasswordEncoder passwordEncoder() {

return new BCryptPasswordEncoder();

}

// Cấu hình CORS toàn cục cho Frontend

@Bean

public CorsConfigurationSource corsConfigurationSource() {

CorsConfiguration configuration = new CorsConfiguration();

configuration.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:4200")); //

configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH"));

configuration.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-Requested-With"));

configuration.setAllowCredentials(true);

UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();

source.registerCorsConfiguration("/", configuration);
```

```
return source;

}

}
```

---

4. KIẾN TRÚC REFRESH TOKEN & CƠ CHẾ ĐĂNG XUẤT AN TOÀN (SAFE LOGOUT)

4.1 Tại sao phải có Refresh Token?

- Nếu **Access Token** có thời hạn quá dài (ví dụ: 30 ngày), khi token bị hacker đánh cắp trên đường truyền, hacker sẽ có toàn quyền truy cập tài khoản suốt 30 ngày mà bạn không thể thu hồi được.
- 👉 **Giải pháp chuẩn ngành ngân hàng/doanh nghiệp:**

  - **Access Token:** Hạn sử dụng **cực ngắn (10 - 15 phút)**. Dùng để gửi kèm mỗi request gọi API.

  - **Refresh Token:** Hạn sử dụng **dài (7 - 30 ngày)**. Lưu trữ mã hóa trong Database (`refresh_tokens`). Chỉ dùng đúng 1 mục đích duy nhất: Gọi endpoint `/api/v1/auth/refresh-token` để xin một Access Token mới khi cái cũ hết hạn.

```
sequenceDiagram

autonumber


actor Client as Frontend (React/Angular)


participant Auth as Auth Server (Spring Boot)


participant DB as Database


Client->>Auth: POST /api/v1/auth/login (Username + Password)


Auth->>DB: Kiểm tra tài khoản


Auth->>Auth: Tạo Access Token (15 phút) & Refresh Token (30 ngày)


Auth->>DB: Lưu Refresh Token vào bảng DB


Auth-->>Client: Trả về { accessToken, refreshToken }


Note over Client,Auth: 15 phút sau... Access Token hết hạn!


Client->>Auth: GET /api/v1/products (Access Token hết hạn)


Auth-->>Client: 401 Unauthorized (Token Expired)


Client->>Auth: POST /api/v1/auth/refresh-token { refreshToken }


Auth->>DB: Kiểm tra Refresh Token có hợp lệ & chưa hết hạn trong DB không?


Auth->>Auth: Tạo Access Token MỚI (15 phút)


Auth-->>Client: Trả về { newAccessToken }
```

```
Client->>Auth: Tiếp tục gọi API bình thường mà người dùng KHÔNG bị văng ra ngoài!
```

4.2 Cơ chế Đăng xuất an toàn (Safe Logout)

Khi người dùng bấm "Đăng xuất" (Logout):

1. Client gửi request: `POST /api/v1/auth/logout` kèm `refreshToken`.

2. Phía Server xóa hoặc đổi trạng thái `revoked = true` của Refresh Token trong Database.

3. Dù hacker có lưu Refresh Token đó thì cũng không bao giờ xin cấp lại Access Token mới được nữa!

---

5. PHÂN QUYỀN TRÊN TỪNG PHƯƠNG THỨC VỚI `@PreAuthorize` (RBAC)

Để bảo vệ từng hàm nghiệp vụ cụ thể, ta sử dụng cú pháp SpEL (Spring Expression Language) trên cấp độ Method:

```
@RestController
@RequestMapping("/api/v1/admin/users")

public class UserAdminController {

// Chỉ có tài khoản có Role là 'ROLE_ADMIN' mới được phép gọi hàm này

@PreAuthorize("hasRole('ADMIN')")

@DeleteMapping("/{id}")

public ResponseEntity<Void> deleteUser(@PathVariable Long id) {

// ...

return ResponseEntity.noContent().build();

}

// Phân quyền nâng cao: Chỉ Admin HOẶC chính chủ sở hữu tài khoản mới được sửa thông tin

@PreAuthorize("hasRole('ADMIN') or #username == authentication.principal.username")

@PutMapping("/{username}")

public ResponseEntity<UserResponse> updateProfile(

@PathVariable String username,

@RequestBody UserProfileRequest request

) {

// ...
```

```
}

}
```

Cơ chế này giúp code bảo mật của bạn cực kỳ linh hoạt, an toàn và dễ đọc.

CHƯƠNG 07: THIẾT KẾ HỆ THỐNG TOÀN DIỆN CHO DỰ ÁN THỰC CHIẾN SMASH ZONE

(Comprehensive System Architecture, Class & Sequence Diagrams for Smash Zone)

---

1. SƠ ĐỒ LỚP HỆ THỐNG (CLASS DIAGRAM) CHUẨN ENTERPRISE SPRING BOOT 3

Dưới đây là thiết kế Sơ đồ Lớp (Class Diagram) đa tầng của hệ thống **Smash Zone Badminton Shop**, thể hiện sự tách biệt rõ ràng giữa 4 tầng: **Entities**, **Repositories**, **Services**, **Controllers** và **DTOs (Java 17 Records)**.

```
classDiagram
    %% Entities
    class User {
        +Integer id
        +String username
        +String password
        +String fullName
        +String email
        +String phone
        +String address
        +String role
        +LocalDateTime createdAt
    }

class Product {
        +Integer id
        +Category category
        +String name
        +Double price
        +String productLink
        +String image
        +Integer quantity
        +Double importPrice
        +String metadataForAi
        +LocalDateTime createdAt
    }


class Category {
        +Integer id
        +String name
        +String description
        +List~Product~ products
    }


class Order {
        +Integer id
        +Integer userId
        +Double totalAmount
        +String status
        +String shippingAddress
        +String paymentMethod
        +LocalDateTime createdAt
        +List~OrderDetail~ orderDetails
    }


class OrderDetail {
        +Integer id
        +Order order
        +Integer productId
        +Integer quantity
        +Double price
    }
```

```
class UserInteraction {
        +Integer id
        +Integer userId
        +Integer productId
        +String interactionType
        +Double interactionScore
        +LocalDateTime createdAt
    }


class ProductRecommendation {
        +Integer id
        +Integer userId
        +Integer productId
        +Double score
        +String reason
        +LocalDateTime createdAt
    }


%% Repositories (Spring Data JPA)
    class UserRepository {
        <<interface>>
        +findByUsername(username)
        +findByEmail(email)
        +existsByUsername(username)
    }


class ProductRepository {
        <<interface>>
        +findByCategoryId(catId)
        +findByCategoryName(catName)
        +searchProducts(keyword)
    }


class OrderRepository {
        <<interface>>
        +findByUserIdOrderByCreatedAtDesc(userId)
        +findAllByOrderByCreatedAtDesc()
    }


%% Services
    class AuthService {
        <<interface>>
        +login(LoginRequest, session) UserResponse
        +register(RegisterRequest) UserResponse
        +logout(session)
        +getCurrentUser(session) User
    }
```

```
class ProductService {
        <<interface>>
        +getAllProducts() List~ProductResponse~
        +getProductById(id) ProductResponse
        +searchProducts(kw) List~ProductResponse~
        +createProduct(ProductCreateRequest) ProductResponse
    }


class OrderService {
        <<interface>>
        +checkout(CheckoutRequest, session) OrderResponse
        +getOrdersByUserId(userId) List~OrderResponse~
        +updateOrderStatus(id, OrderStatusUpdateRequest) OrderResponse
    }


class RecommendationService {
        <<interface>>
        +trackInteraction(userId, productId, type)
        +getRecommendations(userId, limit) List~RecommendationResponse~
    }


%% Controllers
    class AuthController {
        +login(LoginRequest, session)
        +register(RegisterRequest)
        +logout(session)
        +getCurrentUser(session)
    }


class ProductApiController {
        +getProducts(search, category, categoryId)
        +getProductById(id, session)
        +getRecommendations(limit, session)
        +trackInteraction(payload, session)
    }


class OrderApiController {
        +checkout(CheckoutRequest, session)
        +getMyOrders(session)
        +getOrderById(id, session)
    }


Product --> Category : belongs to
    Order "1" -- "" OrderDetail : contains
    ProductApiController --> ProductService : injects
    ProductApiController --> RecommendationService : injects
    OrderApiController --> OrderService : injects
```

```
    AuthController --> AuthService : injects
    ProductService --> ProductRepository : uses
    OrderService --> OrderRepository : uses
    AuthService --> UserRepository : uses
```

---

2. SƠ ĐỒ TUẦN TỰ (SEQUENCE DIAGRAMS) CÁC NGHIỆP VỤ CỐT LÕI

2.1 Luồng 1: Xác Thực & Đăng Nhập Hệ Thống

```
sequenceDiagram
    autonumber
    actor Client as Trình Duyệt (Web Browser)
    participant AC as AuthController
    participant AS as AuthServiceImpl
    participant UR as UserRepository
    participant Sess as HttpSession

Client->>AC: POST /api/auth/login {username, password}
    Note over AC: @Valid kích hoạt kiểm tra rỗng
    AC->>AS: login(LoginRequest, session)
    AS->>UR: findByUsername(username)
    UR-->>AS: Optional<User>



alt Người dùng không tồn tại
        AS-->>AC: Ném BadRequestException("Tên đăng nhập không đúng")
        AC-->>Client: 400 Bad Request {success: false, message: "..."}
    else Người dùng hợp lệ
        AS->>AS: Hash SHA-256(password) & So sánh
        alt Sai mật khẩu
            AS-->>AC: Ném BadRequestException("Mật khẩu không chính xác")
            AC-->>Client: 400 Bad Request
        else Mật khẩu chính xác
            AS->>Sess: setAttribute("userId", user.getId())
            AS->>Sess: setAttribute("user", user)
            AS-->>AC: UserResponse
            AC-->>Client: 200 OK ApiResponse {success: true, data: UserResponse}
        end
    end
```

2.2 Luồng 2: Quy Trình Đặt Hàng & Thanh Toán Nguyên Tử (Atomic Checkout Transaction)

```
sequenceDiagram
    autonumber
    actor Client as Khách Hàng (Client)
    participant OC as OrderApiController
    participant OS as OrderServiceImpl (@Transactional)
    participant CS as CartServiceImpl
    participant PR as ProductRepository
    participant OR as OrderRepository
    participant ODR as OrderDetailRepository
    participant RS as RecommendationServiceImpl

Client->>OC: POST /api/orders/checkout {shippingAddress, paymentMethod}
    OC->>OS: checkout(request, session)
    OS->>CS: getCart(session)
    CS-->>OS: CartResponse (danh sách món, tổng tiền)


loop Duyệt từng sản phẩm trong giỏ hàng
        OS->>PR: findById(productId)
        PR-->>OS: Product entity
        Note over OS: Kiểm tra tồn kho (quantity >= cartQty)
        alt Hết hàng hoặc không đủ tồn kho
            OS-->>OC: Ném BadRequestException("Tồn kho không đủ")
            Note over OS: Spring tự động Rollback toàn bộ giao dịch!
            OC-->>Client: 400 Bad Request
        else Đủ tồn kho
            OS->>PR: setQuantity(quantity - cartQty) & save()
        end
    end


OS->>OR: save(new Order(userId, totalAmount, "PENDING", ...))
    OR-->>OS: savedOrder (có sinh order_id)


loop Lưu chi tiết đơn hàng
        OS->>ODR: save(new OrderDetail(savedOrder, productId, qty, price))
        OS->>RS: trackInteraction(userId, productId, "PURCHASE")
        Note over RS: Ghi nhận tương tác mua hàng (điểm số 5.0)
    end


OS->>CS: clearCart(session)
    Note over CS: Xóa trắng session giỏ hàng
    OS-->>OC: OrderResponse
    OC-->>Client: 200 OK ApiResponse {message: "Đặt hàng thành công", data: OrderResponse}
```

2.3 Luồng 3: Thuật Toán AI Recommendation Engine (Content-Based & Collaborative Filtering)

```
sequenceDiagram
    autonumber
    actor User as Người Dùng
    participant PC as ProductApiController
    participant RS as RecommendationServiceImpl
    participant IR as UserInteractionRepository
    participant PR as ProductRepository
    participant RR as ProductRecommendationRepository

User->>PC: GET /api/recommendations?limit=6
    PC->>RS: getRecommendations(userId, limit)


alt Khách vãng lai (userId == null)
        RS->>PR: findAll()
        PR-->>RS: Danh sách sản phẩm
        Note over RS: Lấy 6 sản phẩm bán chạy / phổ biến nhất
        RS-->>PC: List<RecommendationResponse>
        PC-->>User: 200 OK (Gợi ý thịnh hành)
    else Khách hàng đã đăng nhập
        RS->>IR: findByUserIdOrderByCreatedAtDesc(userId)
        IR-->>RS: Lịch sử tương tác (VIEW: 1.0, CLICK: 2.0, CART: 3.0, PURCHASE: 5.0)
        Note over RS: 1. Tính vector trọng số sản phẩm & phân tích Content-Based<br/>(Category
        RS->>RR: deleteByUserId(userId) & saveAll(topRecommendations)
        RS-->>PC: List<RecommendationResponse>
        PC-->>User: 200 OK (Gợi ý cá nhân hóa)
    end
```

---

3. KẾ HOẠCH DỰ ÁN, PHÂN BỔ THỜI GIAN & SƠ ĐỒ GANTT CHART (6 TUẦN)

3.1 Bảng Phân Rã Công Việc & Thời Gian Thực Hiện (WBS - Work Breakdown Structure)

Dự án được phân bổ trong **6 tuần làm việc (tương đương 36 ngày làm việc tiêu chuẩn)**:

Giai đoạn (Phase)	Tác vụ chi tiết (Task Description)	Thời lượng	Người phụ trách	Kết quả đầu ra (Deliverable)
Giai đoạn 1: Khảo sát & Thiết kế	- Phân tích yêu cầu bài toán Web Shop Cầu Lông. - Thiết kế ERD Database MySQL `badminton_shop`. - Thiết kế Sơ đồ Lớp & Sơ đồ Tuần tự.	Tuần 1 (Ngày 1 - 6)	System Architect / Lead Dev	Bản đặc tả yêu cầu, File SQL DDL, Sơ đồ UML.
Giai đoạn 2: Nền tảng Core & Persistence	- Khởi tạo Spring Boot 3.2.3 trên NetBeans. - Cấu hình HikariCP, JPA & Hibernate. - Viết 7 JPA Entities & 7 JpaRepositories.	Tuần 2 (Ngày 7 - 12)	Backend Developer	Tầng Entity & Repository hoạt động và test kết nối DB.
Giai đoạn 3: DTOs & Service Layer	- Hiện thực DTOs với Java 17 `record`. - Cài đặt AuthService, ProductService, CategoryService. - Viết CartService và OrderService với `@Transactional`.	Tuần 3 (Ngày 13 - 18)	Backend Developer	Tầng Service hoàn chỉnh, xử lý transaction an toàn.
Giai đoạn 4: REST Controllers & Validation	- Viết các RestControllers (`/api/auth`, `/api/products`, `/api/cart`, `/api/orders`, `/api/admin`). - Bổ sung Bean Validation (`@Valid`). - Viết `GlobalExceptionHandler` với `@RestControllerAdvice`.	Tuần 4 (Ngày 19 - 24)	API Developer	Bộ REST APIs hoàn chỉnh, trả về chuẩn JSON RFC 7807 / ApiResponse.
Giai đoạn 5: AI Engine & Tích hợp Frontend	- Cài đặt thuật toán Gợi ý Content-Based & Collaborative. - Tích hợp HTML5, CSS3, Vanilla JS Fetch API. - Hoàn thiện giao diện Admin Dashboard và Giỏ hàng.	Tuần 5 (Ngày 25 - 30)	Fullstack Developer	Website chạy mượt mà, gợi ý sản phẩm hiển thị real-time.
Giai đoạn 6: Kiểm thử & Đóng gói Nghiệm thu	- Viết Unit Test & Integration Test. - Tối ưu cấu hình Docker & NetBeans Run Action. - Hoàn thiện tài liệu kỹ thuật & Báo cáo đồ án.	Tuần 6 (Ngày 31 - 36)	QA / Tech Lead	Hệ thống chạy ổn định, File PDF báo cáo hoàn chỉnh.

3.2 Sơ Đồ Tiến Độ Dự Án (Mermaid Gantt Chart)

```
gantt
    title KẾ HOẠCH TIẾN ĐỘ THỰC HIỆN DỰ ÁN SMASH ZONE (6 TUẦN)
    dateFormat  YYYY-MM-DD
    section Giai đoạn 1: Khảo sát & Thiết kế
    Khảo sát yêu cầu & Đặc tả chức năng       :done,    des1, 2026-09-01, 2d
    Thiết kế CSDL MySQL (ERD & DDL)           :done,    des2, after des1, 2d
    Thiết kế Kiến trúc Spring Boot & UML      :done,    des3, after des2, 2d

section Giai đoạn 2: Nền tảng & Persistence
    Khởi tạo Spring Boot 3 & Maven            :done,    core1, 2026-09-07, 2d
    Viết JPA Entities & Ánh xạ quan hệ        :done,    core2, after core1, 2d
    Viết JpaRepository & Derived Queries      :done,    core3, after core2, 2d


section Giai đoạn 3: DTOs & Service Layer
    Thiết kế Java 17 Records DTOs             :done,    srv1, 2026-09-13, 2d
    Cài đặt AuthService & Hash Mật khẩu       :done,    srv2, after srv1, 2d
    Cài đặt Product, Cart & OrderService      :done,    srv3, after srv2, 2d


section Giai đoạn 4: REST APIs & Validation
    Xây dựng AuthController & ProductController:done,    api1, 2026-09-19, 2d
    Xây dựng Cart, Order & AdminController    :done,    api2, after api1, 2d
    Bean Validation & GlobalExceptionHandler  :done,    api3, after api2, 2d


section Giai đoạn 5: AI Engine & Frontend
    Hiện thực Thuật toán Gợi ý Sản phẩm AI    :done,    fe1, 2026-09-25, 3d
    Ghép giao diện HTML/CSS/JS với REST API   :done,    fe2, after fe1, 3d


section Giai đoạn 6: Kiểm thử & Triển khai
    Kiểm thử chức năng & Tải cao              :active,  tst1, 2026-10-01, 3d
    Đóng gói Docker & Nghiệm thu đồ án        :         tst2, after tst1, 3d
```

---

4. HƯỚNG DẪN VẬN HÀNH & CHẠY DỰ ÁN TRÊN APACHE NETBEANS IDE

Để mở và chạy dự án **Smash Zone** trực tiếp trên **Apache NetBeans IDE**, lập trình viên thực hiện theo 3 bước chuẩn xác sau:

Bước 1: Mở Dự án trong Apache NetBeans

1. Khởi động **Apache NetBeans IDE** (phiên bản 18+ trở lên). 2. Vào menu **File -> Open Project...** (hoặc phím tắt `Ctrl + Shift + O`). 3. Điều hướng tới thư mục `c:\Users\Dell\TTCS_WEB - Sao chép`. 4. NetBeans sẽ nhận diện biểu tượng dự án Maven có hình lá mầm Spring Boot. Chọn thư mục và nhấn **Open Project**.

Bước 2: Chuẩn bị Cơ sở Dữ liệu MySQL

1. Khởi động dịch vụ MySQL Server (qua XAMPP, MySQL Workbench hoặc Windows Services). 2. Đảm bảo Database mang tên `badminton_shop` đã tồn tại trên cổng `3306` :

```
CREATE DATABASE IF NOT EXISTS badminton_shop CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_
```

3. Kiểm tra thông tin tài khoản trong file `src/main/resources/application.properties` : - `spring.datasource.username=root` - `spring.datasource.password=123456`

Bước 3: Nhấn Chạy Dự Án (Run Project)

1. Trong cửa sổ **Projects** bên trái NetBeans, nhấp chuột phải vào dự án **smashzone** -> chọn **Run** (hoặc nhấn phím tắt `F6`). 2. NetBeans sẽ kích hoạt Maven thực hiện lệnh `spring-boot:run` . 3. Quan sát cửa sổ Output bên dưới, khi xuất hiện thông báo:

```
Tomcat started on port 8080 (http) with context path ''
Started SmashZoneApplication in X.XXX seconds
```

4. Mở trình duyệt Web truy cập vào địa chỉ: - **Giao diện Cửa hàng:** `http://localhost:8080/index.html` - **Tài khoản Khách hàng:** Đăng ký tài khoản mới hoặc dùng tài khoản có sẵn

- **Tài khoản Quản trị viên (Admin):** Username: `admin` Password: `1` -> Truy cập `http://localhost:8080/admin/dashboard.html`

CHƯƠNG 08: BẢN THIẾT KẾ DỰ ÁN THỰC CHIẾN & QUY TRÌNH TỰ XÂY DỰNG TỪ A-Z

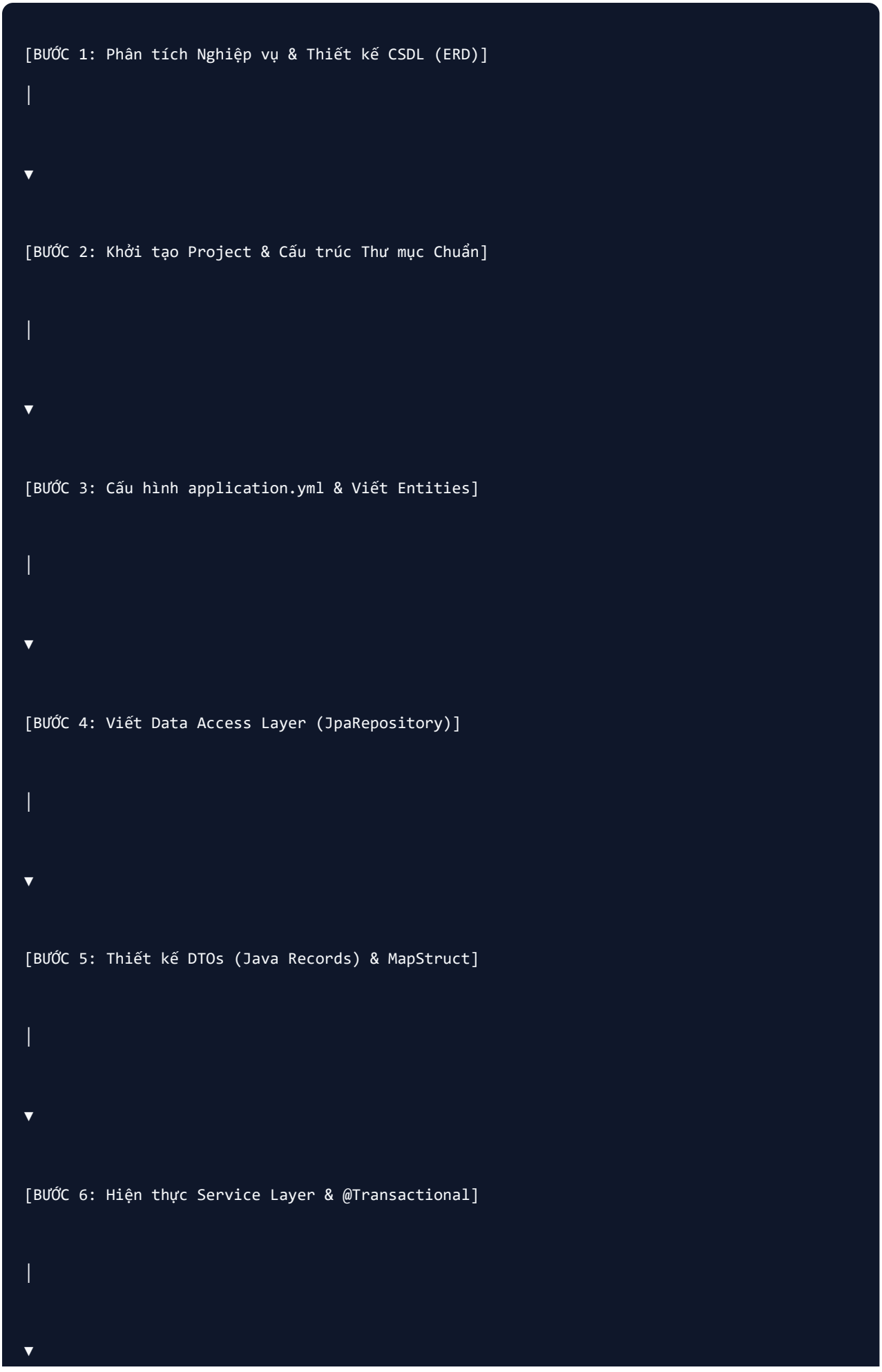
(Step-by-Step Blueprint to Build Any Project, Architecture Case Studies & Docker Deployment)

---

1. QUY TRÌNH CHUẨN 8 BƯỚC XÂY DỰNG DỰ ÁN MỚI TỪ CON SỐ 0

(The 8-Step Blueprint to Build Any Enterprise Project from Scratch)

Khi bạn được giao một đề tài mới (Website Bán hàng, Ứng dụng Quản lý Khách sạn, Mạng xã hội, Đặt vé xe, App Giao đồ ăn...), hãy mở đúng quy trình 8 bước này ra và thực hiện lần lượt. Đây là quy trình chuẩn mực được áp dụng tại các tập đoàn công nghệ lớn:


[BƯỚC 1: Phân tích Nghiệp vụ & Thiết kế CSDL (ERD)]
[BƯỚC 2: Khởi tạo Project & Cấu trúc Thư mục Chuẩn]
[BƯỚC 3: Cấu hình application.yml & Viết Entities]
[BƯỚC 4: Viết Data Access Layer (JpaRepository)]
[BƯỚC 5: Thiết kế DTOs (Java Records) & MapStruct]
[BƯỚC 6: Hiện thực Service Layer & @Transactional]

```
[BƯỚC 7: Viết Controller, Validation & ControllerAdvice]

|

▼

[BƯỚC 8: Khóa Bảo mật Spring Security, JWT & Dockerize]
```

🔹 BƯỚC 1: Phân tích Nghiệp vụ & Vẽ Sơ đồ Quan hệ Thực thể (ERD)

- Liệt kê các đối tượng cốt lõi của hệ thống.
- Xác định mối quan hệ giữa chúng (1-1, 1-N, N-N).
- *Quy tắc:* Luôn xác định bảng nào chứa khóa ngoại (Foreign Key) trước khi viết code.

🔹 BƯỚC 2: Khởi tạo Project & Thiết lập Cấu trúc Package chuẩn

1. Truy cập `https://start.spring.io/`:

- Project: **Maven**

- Language: **Java 17** (hoặc 21)

- Spring Boot: **3.2.x** trở lên

- Dependencies: `Spring Web`, `Spring Data JPA`, `PostgreSQL Driver` (hoặc MySQL), `Validation`, `Lombok`, `Spring Security`, `Docker Compose Support`.

2. Tổ chức cấu trúc thư mục Package theo chuẩn **Layered / Feature-based Architecture**:

```
com.myproject.app/
├── config/             # Cấu hình Spring Beans, OpenAPI, Security, Async, Auditing
├── controller/         # Tiếp nhận HTTP Request, gọi Service, trả về ResponseEntity
├── dto/                # Các Java Record chứa dữ liệu trao đổi (Request/Response)
├── entity/             # Các JPA Entity ánh xạ trực tiếp với bảng Database
├── exception/          # Custom Exceptions & GlobalExceptionHandler (@RestControllerAdvice)
├── mapper/             # Các Interface MapStruct chuyển đổi Entity <-> DTO
├── repository/         # Các Interface kế thừa JpaRepository để truy vấn DB
├── security/           # JwtService, JwtFilter, UserDetailsServiceImpl
└── service/            # Business Logic, tính toán, quản lý @Transactional
```

🔹 BƯỚC 3: Cấu hình `application.yml` & Viết Entities

- Thiết lập chuỗi kết nối Database với `HikariCP` connection pool.
- Tạo Entity: Luôn nhớ `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Enumerated(EnumType.STRING)`.
- Kích hoạt JPA Auditing để tự động lưu ngày tạo (`@CreatedDate`).

🔹 BƯỚC 4: Viết Data Access Layer (`JpaRepository`)

- Tạo các interface kế thừa `JpaRepository`.
- Viết các hàm Derived Query (`findByEmail`, `existsBySku`).
- Nếu có quan hệ nhiều bảng, nhớ thêm `JOIN FETCH` hoặc `@EntityGraph` để tránh lỗi N+1 Query.

🔹 BƯỚC 5: Thiết kế DTOs (`record`) & MapStruct Mapper

- Tuyệt đối không để Entity lọt ra ngoài Controller.
- Tạo các DTO riêng cho từng hành động: `CreateUserRequest` , `UpdateUserRequest` , `UserResponse` .
- Khai báo MapStruct Mapper để tự động chuyển đổi dữ liệu.

🔹 BƯỚC 6: Hiện thực Service Layer & Business Logic

- Đánh dấu `@Service` và `@RequiredArgsConstructor` .
- Thêm `@Transactional(rollbackFor = Exception.class)` cho các hàm thêm/sửa/xóa.
- Thêm `@Transactional(readOnly = true)` cho các hàm tìm kiếm/đọc.
- Ném ra các Custom Exception rõ nghĩa (`ResourceNotFoundException` , `DuplicateResourceException`) khi dữ liệu vi phạm quy tắc.

🔹 BƯỚC 7: Viết Controller, Validation & Global Error Handler

- Sử dụng `@RestController` và `@RequestMapping` .
- Đặt `@Valid` trên `@RequestBody` của hàm tiếp nhận dữ liệu.
- Hoàn thiện `@RestControllerAdvice` để bắt gọn mọi lỗi và trả về chuẩn JSON RFC 7807 `ProblemDetail` .

🔹 BƯỚC 8: Khóa Bảo mật (Spring Security, JWT) & Đóng gói Docker

- Tạo `JwtService` , `JwtAuthenticationFilter` và cấu hình `SecurityFilterChain` .
- Bật mã hóa mật khẩu `BCryptPasswordEncoder` .
- Viết `Dockerfile` và `docker-compose.yml` để chạy ứng dụng trên bất kỳ máy chủ nào chỉ với 1 câu lệnh.

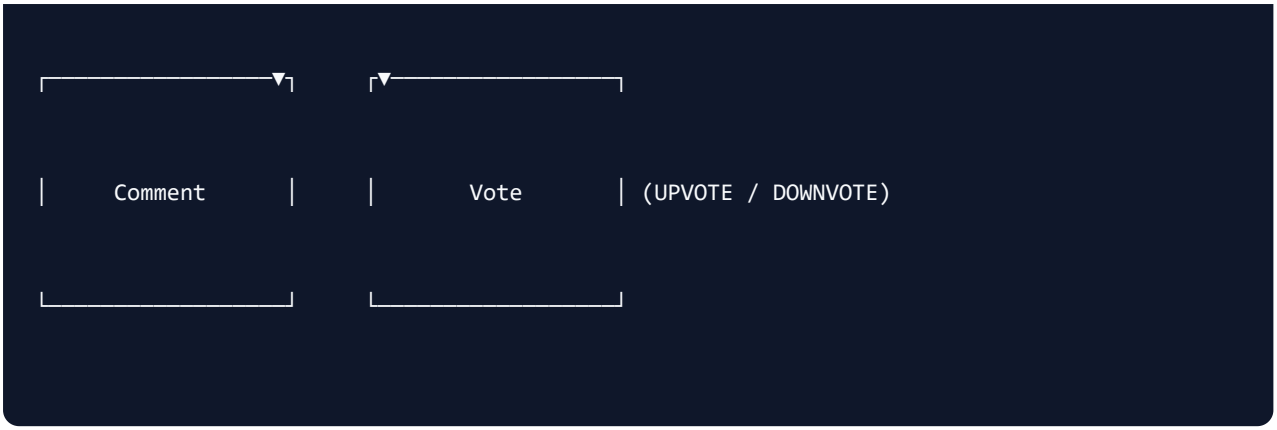
---

2. PHÂN TÍCH KIẾN TRÚC 2 DỰ ÁN TIÊU BIỂU TỪ KHÓA HỌC

2.1 Dự án 1: Reddit Clone Enterprise Backend (Video 6)

Dự án này là minh chứng tiêu chuẩn cho một hệ thống mạng xã hội thu nhỏ hoàn chỉnh:


User
1
N
Subreddit
(Diễn đàn con, ví dụ: r/programming)
1
N
Post
(Bài viết)
1
1
N
N



Các kỹ thuật đỉnh cao được áp dụng trong Reddit Clone:

1. **Xử lý bình chọn (Voting System):**

- Bảng `Vote` có quan hệ `@ManyToOne` với `Post` và `User`.

- Một người dùng chỉ được phép bình chọn 1 lần (UP hoặc DOWN). Nếu đã UP mà bấm UP lần nữa thì hủy vote; nếu chuyển sang DOWN thì đảo ngược số điểm. Toàn bộ logic kiểm tra và cập nhật `post.setVoteCount()` được bọc kín trong một `@Transactional` an toàn tuyệt đối.

2. **Kích hoạt tài khoản bất đồng bộ:**

- Khi đăng ký, sinh một `VerificationToken` có hạn 24 giờ lưu vào DB.

- Bắn `@Async` gọi `MailService` gửi email chứa link xác thực: `https://myreddit.com/api/v1/auth/accountVerification/{token}`.

- Người dùng click link ➔ Server kích hoạt `user.setEnabled(true)`.

3. **Quản lý Token kép:** Kết hợp hoàn hảo giữa `AccessToken` (15 phút) và `RefreshToken` (có lưu Database để thu hồi khi Logout).

2.2 Dự án 2: Fullstack Movie Review App (Spring Boot + React + MongoDB) (Video 4)

Dự án này minh họa khả năng kết nối linh hoạt của Spring Boot với cơ sở dữ liệu phi quan hệ (NoSQL Document Store):

- **Backend:** Spring Boot 3 + Spring Data MongoDB.
- **Database:** MongoDB Atlas (Cloud NoSQL Database).
- **Frontend:** React SPA (Hooks: `useState`, `useEffect`, `Axios`) + Bootstrap UI.
- **Bài học cốt lõi:**

  - Ánh xạ Document lồng nhau: Một bộ phim (`Movie`) chứa một mảng tham chiếu `@DocumentReference List reviewIds`.

  - Giải quyết bài toán **CORS (Cross-Origin Resource Sharing)**: Mở cổng an toàn cho phép React (chạy trên port 3000) gọi sang Spring Boot (chạy trên port 8080) mà không bị trình duyệt chặn.

3. BỘ KHUNG XƯƠNG DỰ ÁN MẪU (STARTER SKELETON BOILERPLATE)

Bạn có thể copy trực tiếp các file mẫu này để làm nền tảng khởi đầu cho bất kỳ dự án mới nào của bạn:

3.1 File `pom.xml` chuẩn Enterprise đầy đủ mọi thư viện:

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0

<modelVersion>4.0.0</modelVersion>

<parent>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-parent</artifactId>

<version>3.2.5</version>

<relativePath/>

</parent>

<groupId>com.example</groupId>

<artifactId>spring-boot-master-starter</artifactId>

<version>1.0.0</version>

<name>spring-boot-master-starter</name>

<description>Enterprise Spring Boot Starter Boilerplate</description>

<properties>
```

```
<java.version>17</java.version>

<jjwt.version>0.12.5</jjwt.version>

<mapstruct.version>1.5.5.Final</mapstruct.version>

<lombok.version>1.18.30</lombok.version>

<springdoc.version>2.5.0</springdoc.version>

</properties>

<dependencies>

<!-- 1. Web & REST API -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

<!-- 2. Data Access & JPA -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>
```

```
<!-- 3. PostgreSQL Database Driver (hoặc đổi sang mysql-connector-j) -->

<dependency>

<groupId>org.postgresql</groupId>

<artifactId>postgresql</artifactId>

<scope>runtime</scope>

</dependency>

<!-- 4. Validation (Jakarta Validation) -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-validation</artifactId>

</dependency>

<!-- 5. Security & JWT -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-security</artifactId>

</dependency>

<dependency>
```

```
<groupId>io.jsonwebtoken</groupId>

<artifactId>jjwt-api</artifactId>

<version>${jjwt.version}</version>

</dependency>

<dependency>

<groupId>io.jsonwebtoken</groupId>

<artifactId>jjwt-impl</artifactId>

<version>${jjwt.version}</version>

<scope>runtime</scope>

</dependency>

<dependency>

<groupId>io.jsonwebtoken</groupId>

<artifactId>jjwt-jackson</artifactId>

<version>${jjwt.version}</version>

<scope>runtime</scope>

</dependency>

<!-- 6. Thư viện tiện ích: Lombok & MapStruct -->
```

```
<dependency>

<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

<optional>true</optional>

</dependency>

<dependency>

<groupId>org.mapstruct</groupId>

<artifactId>mapstruct</artifactId>

<version>${mapstruct.version}</version>

</dependency>

<!-- 7. Tài liệu API Swagger / OpenAPI 3 -->

<dependency>

<groupId>org.springdoc</groupId>

<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>

<version>${springdoc.version}</version>

</dependency>

<!-- 8. Giám sát hệ thống Actuator -->
```

```
<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-actuator</artifactId>

</dependency>

<!-- 9. Testing -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-test</artifactId>

<scope>test</scope>

</dependency>

<dependency>

<groupId>org.springframework.security</groupId>

<artifactId>spring-security-test</artifactId>

<scope>test</scope>

</dependency>

<dependency>

<groupId>com.h2database</groupId>
```

```
<artifactId>h2</artifactId>

<scope>test</scope>

</dependency>

</dependencies>

<build>

<plugins>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<configuration>

<source>${java.version}</source>

<target>${java.version}</target>

<annotationProcessorPaths>

<path>

<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

<version>${lombok.version}</version>
```

```
</path>

<path>

<groupId>org.projectlombok</groupId>

<artifactId>lombok-mapstruct-binding</artifactId>

<version>0.2.0</version>

</path>

<path>

<groupId>org.mapstruct</groupId>

<artifactId>mapstruct-processor</artifactId>

<version>${mapstruct.version}</version>

</path>

</annotationProcessorPaths>

</configuration>

</plugin>

<plugin>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-maven-plugin</artifactId>
```

```
</plugin>

</plugins>

</build>

</project>
```

3.2 File cấu hình chuẩn mực `application.yml`:

```
server:

port: 8080


servlet:


context-path: /api/v1


spring:


application:


name: enterprise-backend-app


# Cấu hình kết nối Cơ sở dữ liệu PostgreSQL


datasource:


url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:my_app_db}


username: ${DB_USER:postgres}


password: ${DB_PASSWORD:postgres}


driver-class-name: org.postgresql.Driver


hikari:


maximum-pool-size: 10


minimum-idle: 5


idle-timeout: 300000
```

```
connection-timeout: 20000

# Cấu hình Hibernate JPA

jpa:

hibernate:

ddl-auto: update # Trong môi trường Production đổi thành 'validate'

show-sql: true

properties:

hibernate:

format_sql: true

dialect: org.hibernate.dialect.PostgreSQLDialect
```

```
app:

jwt:

# Chuỗi bí mật 256-bit Base64

secret: 404E635266556A586E3272357538782F413F4428472B4B6250645367566B5970

access-token-expiration-ms: 900000     # 15 phút
```

```
  refresh-token-expiration-ms: 604800000 # 7 ngày
```

```
management:

endpoints:

web:

exposure:

include: health, info, metrics

endpoint:

health:

show-details: always
```

---

4. ĐÓNG GÓI & TRIỂN KHAI PRODUCTION VỚI DOCKER

Để đưa dự án Spring Boot lên môi trường Production (AWS EC2, DigitalOcean, VPS, Kubernetes), ta sử dụng **Docker**.

4.1 `Dockerfile` Multi-Stage Build siêu nhẹ (Tối ưu từ 600MB xuống 130MB)

Tạo file có tên `Dockerfile` tại thư mục gốc của dự án:

```
```

```
FROM maven:3.9.6-eclipse-temurin-17-alpine AS builder

WORKDIR /app
```

```
COPY pom.xml .

RUN mvn dependency:go-offline -B
```

```
COPY src ./src

RUN mvn clean package -DskipTests
```

```
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app
```

```
RUN addgroup -S spring && adduser -S spring -G spring


USER spring:spring
```

```
COPY --from=builder /app/target/*.jar app.jar
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["java", "-XX:+UseContainerSupport", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"
```

4.2 File `docker-compose.yml` (Chạy cả Database + App chỉ với 1 lệnh)

Tạo file `docker-compose.yml` tại thư mục gốc:

```
version: '3.8'

services:

# 1. Dịch vụ Cơ sở dữ liệu PostgreSQL

postgres-db:

image: postgres:16-alpine

container_name: postgres-db

restart: always

environment:

POSTGRES_DB: my_app_db

POSTGRES_USER: postgres

POSTGRES_PASSWORD: mysecretpassword

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s
```

```
timeout: 5s

retries: 5

# 2. Dịch vụ Spring Boot Backend

backend-app:

build:

context: .

dockerfile: Dockerfile

container_name: spring-boot-backend

restart: always

ports:

- "8080:8080"

environment:

DB_HOST: postgres-db

DB_PORT: 5432

DB_NAME: my_app_db

DB_USER: postgres

DB_PASSWORD: mysecretpassword
```

```
SPRING_PROFILES_ACTIVE: prod

depends_on:

postgres-db:

condition: service_healthy

volumes:

postgres_data:
```

🚀 **Cách khởi chạy toàn bộ hệ thống trên VPS / Máy cá nhân:**

Mở terminal tại thư mục dự án và chỉ cần gõ đúng một câu lệnh:

```
docker compose up -d --build
```

Hệ thống sẽ tự động:

1. Kéo image PostgreSQL về, khởi tạo database `my_app_db`.

2. Biên dịch code Java, đóng gói thành Docker Image.

3. Tự động đợi database khởi động khỏe mạnh rồi mới kích hoạt Spring Boot App kết nối vào.

4. Bạn có thể mở trình duyệt gõ: `http://localhost:8080/api/v1/swagger-ui.html` để trải nghiệm hệ thống hoạt động hoàn hảo!

🎯 LỜI KẾT & TÂM THẾ KỸ SƯ

Đến đây, bạn đã nắm giữ toàn bộ các mảnh ghép kỹ thuật từ cơ bản đến đỉnh cao nhất của hệ sinh thái Spring Boot. Khi gặp bất kỳ đề tài hay dự án mới nào, hãy tự tin:

- **Tư duy kiến trúc trước:** Vẽ ERD, định hình DTOs và luồng nghiệp vụ.
- **Tuân thủ Clean Architecture:** Tách biệt rõ Controller, Service, Repository, không để rò rỉ Entity.
- **Bảo mật ngay từ đầu:** Luôn mã hóa mật khẩu, sử dụng JWT Token và phân quyền chặt chẽ.
- **Viết kiểm thử tự động:** Đảm bảo code của bạn không bao giờ sinh ra lỗi hồi quy.

Chúc bạn thành công trên con đường trở thành một Kỹ sư Backend Spring Boot xuất sắc!

thiết lập lại chuẩn form style như này được không á

The current local time is: 2026-09-05T20:26:08+07:00. The user changed setting `Model Selection` from Gemini 3.6 Flash (High) to Gemini 3.8 Flash (High). No need to comment on this change if the user doesn't ask about it. If reporting what model you are, please use a human readable name instead of the exact string.